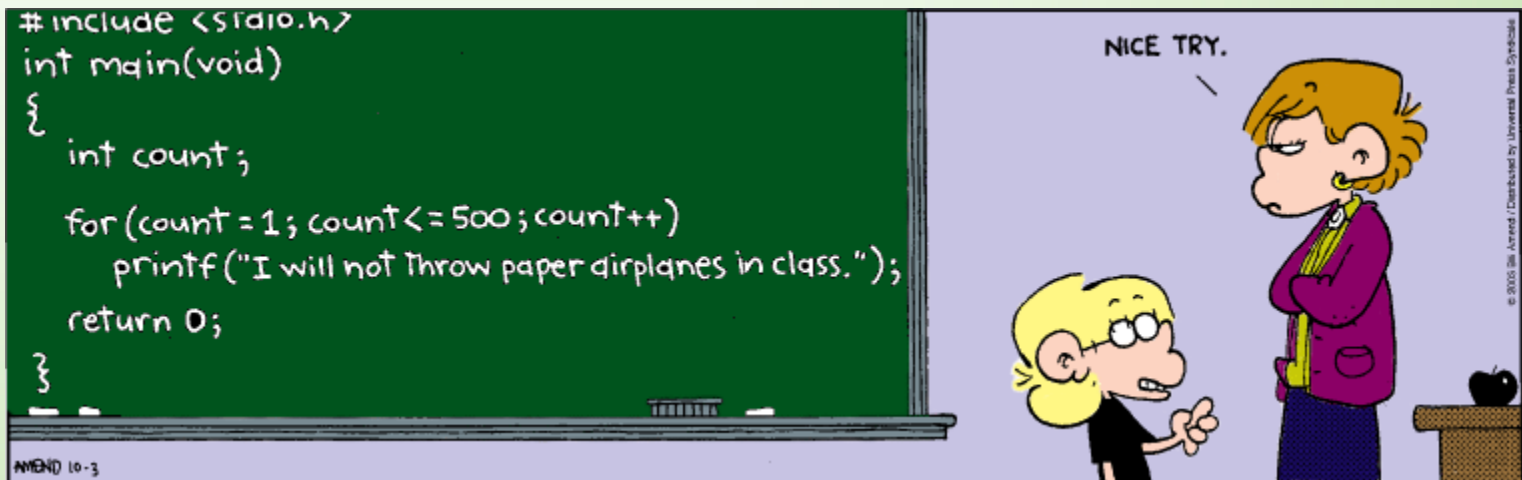


Chapter 6:

Methods





Objectives

- To define methods with formal parameters (§6.2).
- To invoke methods with actual parameters (i.e., arguments) (§6.2).
- To define methods with a return value (§6.3).
- To define methods without a return value (§6.4).
- To pass arguments by value (§6.5).
- To develop reusable code that is modular, easy to read, easy to debug, and easy to maintain (§6.6).
- To write a method that converts hexadecimal to decimals (§6.7).
- To use method overloading and understand ambiguous overloading (§6.8).
- To determine the scope of variables (§6.9).
- To apply the concept of method abstraction in software development (§6.10).
- To design and implement methods using stepwise refinement (§6.10).



Program 1: Sum Many Numbers

- ♦ Write a program that will sum three sets of numbers and then display the sum of each:
 - sum of integers from 1 to 10
 - sum of integers from 20 to 37
 - sum of integers from 35 to 49
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 1: Sum Many Numbers

♦ Step 1: Problem-solving Phase

– Algorithm:

- ♦ This program is really easy
- ♦ For each set of numbers:
 - make a variable sum
 - make a for loop and sum from the first number to the second number
 - print the final sum
- ♦ So this is very easy to do
- ♦ Unfortunately, we have to do it three times!



Program 1: Sum Many Numbers

♦ Step 2: Implementation

```
public class SumManyNumbers{  
    public static void main(String[] args) {  
        int sum = 0;  
        for (int i = 1; i <= 10; i++)  
            sum += i;  
        System.out.println("Sum from 1 to 10 is " + sum);  
  
        sum = 0;  
        for (int i = 20; i <= 37; i++)  
            sum += i;  
        System.out.println("Sum from 20 to 37 is " + sum);  
  
        sum = 0;  
        for (int i = 35; i <= 49; i++)  
            sum += i;  
        System.out.println("Sum from 35 to 49 is " + sum);  
    }  
}
```



Program 1: Sum Many Numbers

♦ Observation

- Each sum is doing something very similar
- In fact, each sum is essentially doing the same thing
- The only difference is the range of numbers
 - ♦ the starting and ending numbers of the sum
- So why do we *repeat* our code three times?
- Wouldn't it be nice if we could write "common" code and then reuse it when needed?
 - ♦ That would be PERFECT!
- This is the idea of methods!



Program 1: Sum Many Numbers

♦ Step 2: Implementation

```
1  public static int sum(int i1, int i2) {  
2      int result = 0;  
3      for (int i = i1; i <= i2; i++)  
4          result += i;  
5  
6      return result;  
7  }  
8  
9  public static void main(String[] args) {  
10     System.out.println("Sum from 1 to 10 is " + sum(1, 10));  
11     System.out.println("Sum from 20 to 37 is " + sum(20, 37));  
12     System.out.println("Sum from 35 to 49 is " + sum(35, 49));  
13 }
```

- ♦ Here, we write a method to calculate the sum
 - And then, inside main, we call/invoke the method three times
- ♦ You don't need to understand this perfectly right now
- ♦ We will spend the next two weeks understanding it!



Methods

♦ What is a method?

- A method is a collection of statements grouped together to perform an operation.
- Guess what?
 - ♦ You have already used many predefined methods!
 - ♦ Examples:

<code>System.out.println</code>	<code>System.exit</code>
<code>Math.pow</code>	<code>Math.random</code>
<code>input.nextInt</code>	<code>"some string".toUpperCase</code>

- ♦ These methods are defined in the Java library
- ♦ In this chapter, you learn how to create your own methods!



Defining a Method

♦ A method consists of:

- method name
- parameters
- return value type
- Body

♦ Syntax:

```
modifier returnType methodName(list of parameters) {  
    // Method body  
}
```



Defining a Method

♦ Sample method:

- We will now look at a sample method
- This method is VERY easy
 - ♦ Given two integers, find the larger value
- Although the logic is easy, we will study this sample method in detail
- We need to understand the anatomy of a method
 - ♦ anatomy: a study of the structure or internal workings of something
 - ♦ In summary: we need to fully understand all components of the method and how it works!



Defining a Method

♦ Sample method:

- Given two integers, determine the maximum value

Define a method

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Invoke a method

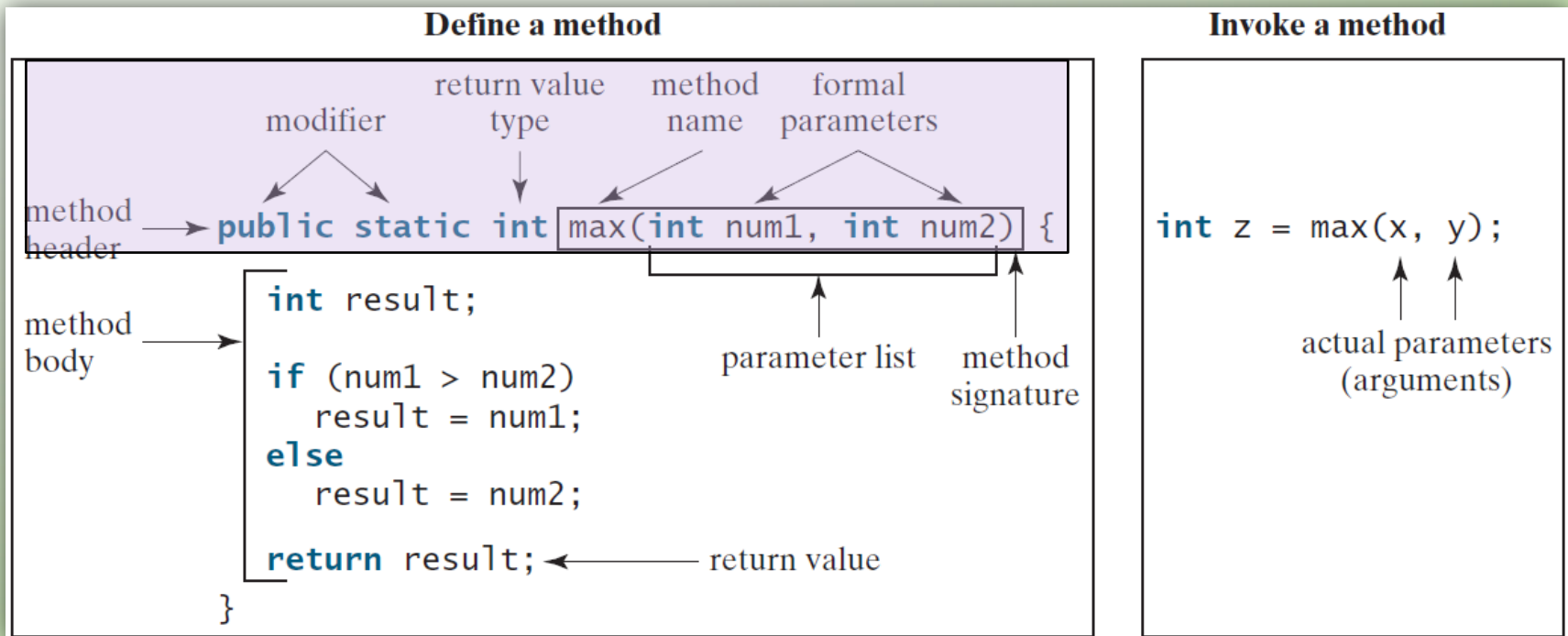
```
int z = max(x, y);
```



Defining a Method

♦ Method Header:

- Specifies the modifiers, return value type, method name, and parameters of the method

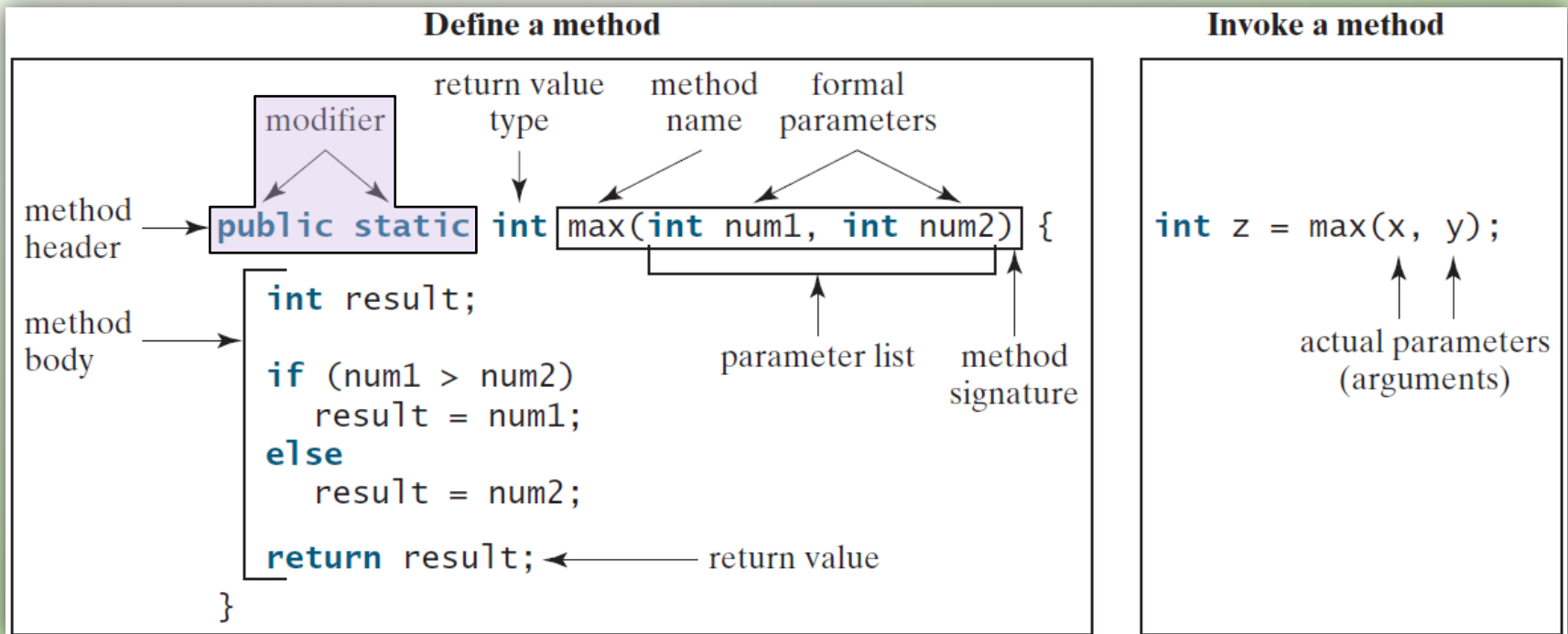




Defining a Method

♦ Modifiers:

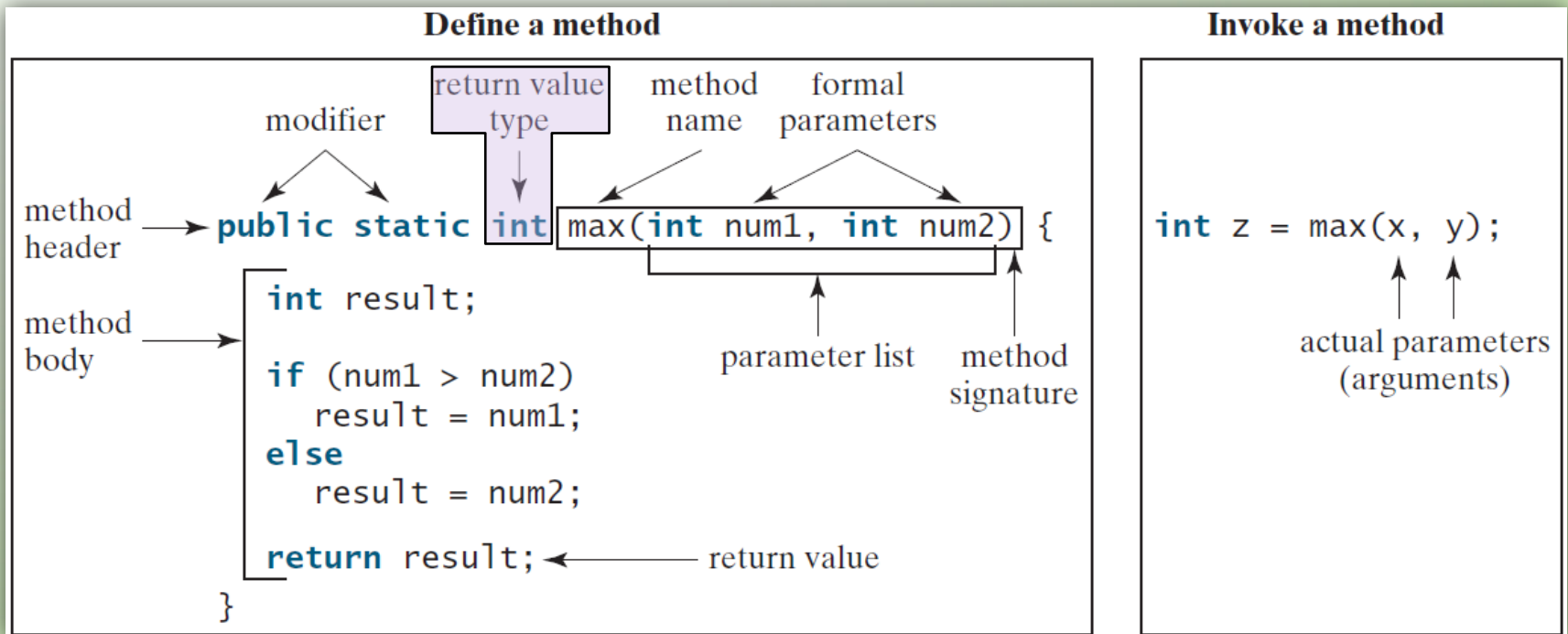
- The **static** modifier is used for all methods in 202
 - ♦ the reason will be discussed later



Defining a Method

♦ Return Value:

- Some methods compute a value and return it
 - ♦ **returnValueType** is the data type of the return value





Defining a Method

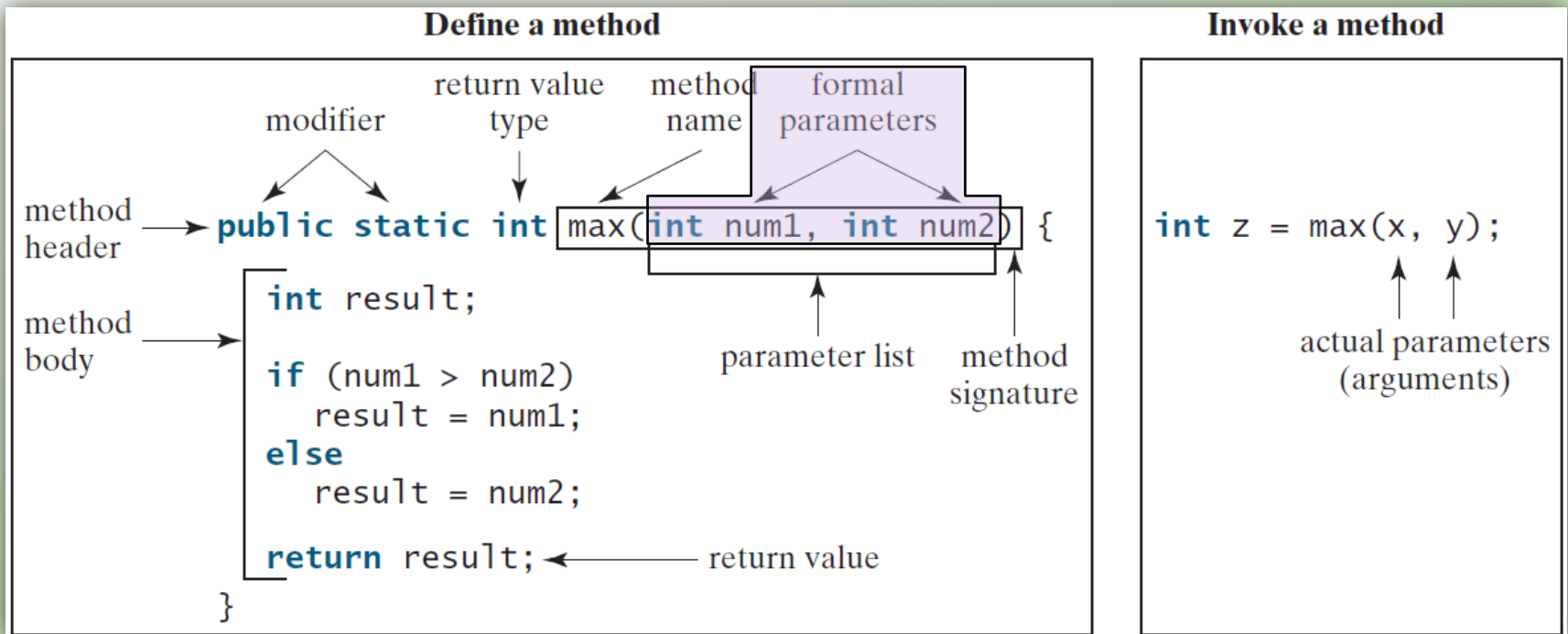
♦ Return Value:

- Some methods compute a value and return it
 - ♦ **returnValueType** is the data type of the return value
- Other methods perform operations but do not return a value
 - ♦ In this case, **returnValueType** is the keyword **void**.
 - ♦ Example:
 - The **returnValueType** is **void** in the **main** method.
- If a method returns a value, it is called a value-returning method
- Else, it is called a void method

Defining a Method

♦ Formal Parameters:

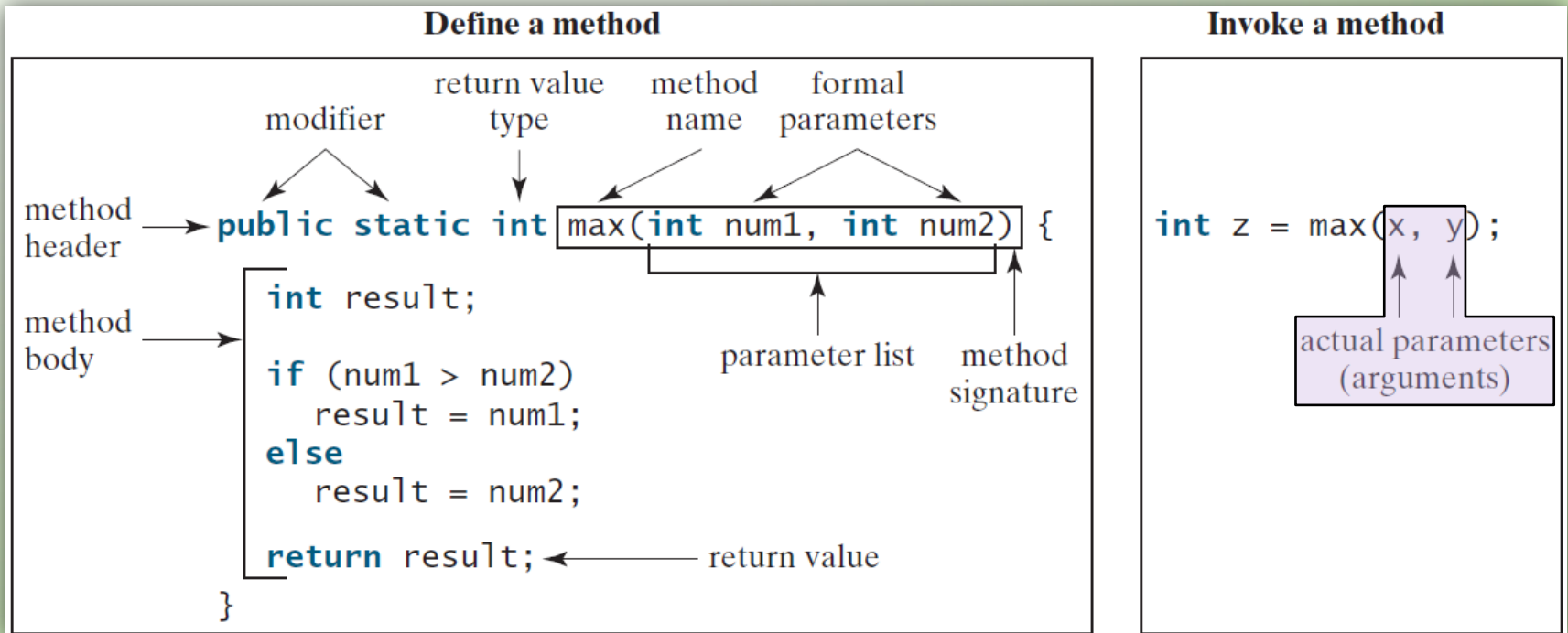
- These are the variables defined in the method header
 - ♦ Think of formal parameters as placeholders



Defining a Method

♦ Actual Parameters:

- When a method is invoked, you must send actual values, which are then saved as formal parameters





Defining a Method

♦ Actual Parameters:

- When a method is invoked, you must send actual values, which are then saved as formal parameters
- These actual (real) values are called actual parameters
- Note:
 - ♦ You can use the word "parameters" or the word "arguments"
 - ♦ BOTH are well-known
- The parameter list (or the argument list) refers to the method's type, order, and number of parameters



Defining a Method

♦ Parameters:

- The parameter list (or the argument list) refers to the method's type, order, and number of parameters
- Parameters are optional
- This means that some methods may have no parameters
- In fact, you know some methods already, which have no parameters:

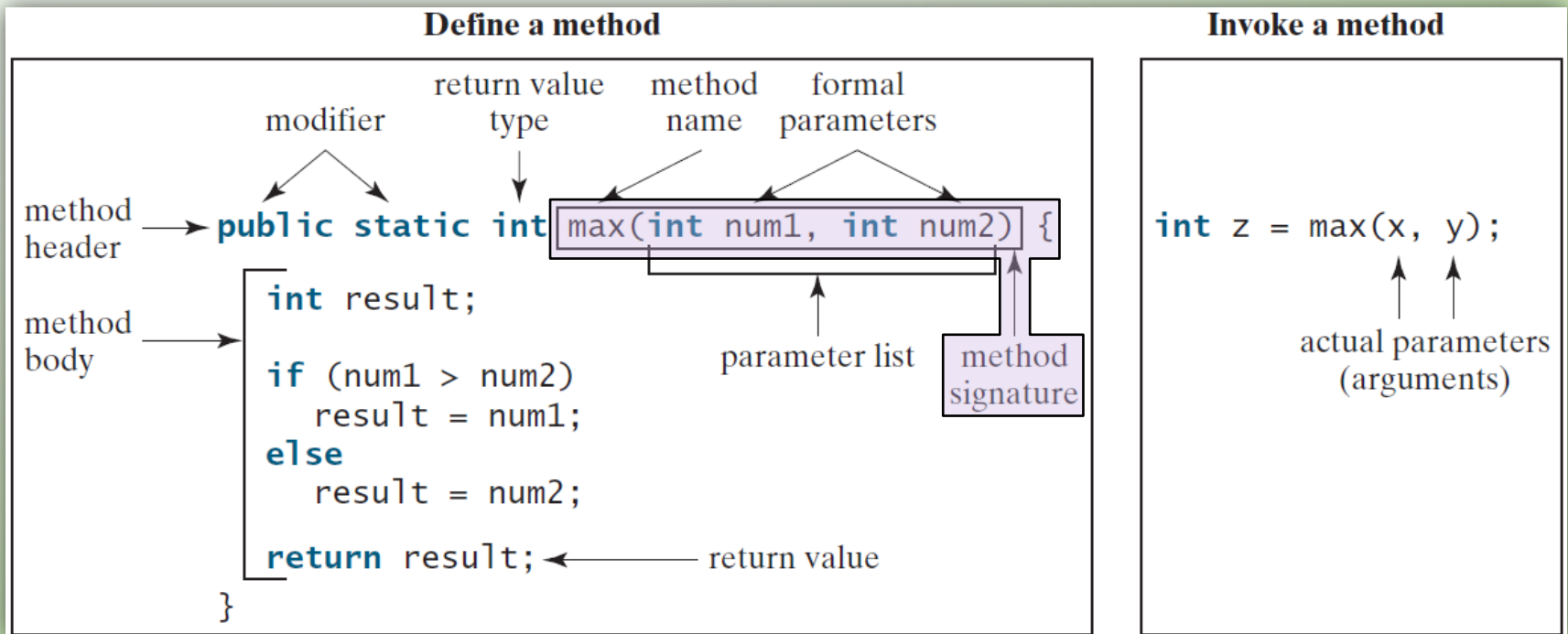
♦ `Math.random()`

- There are no parameters for the `Math.random()` method

Defining a Method

♦ Method Signature:

- The method name and the parameter list together constitute the *method signature*.

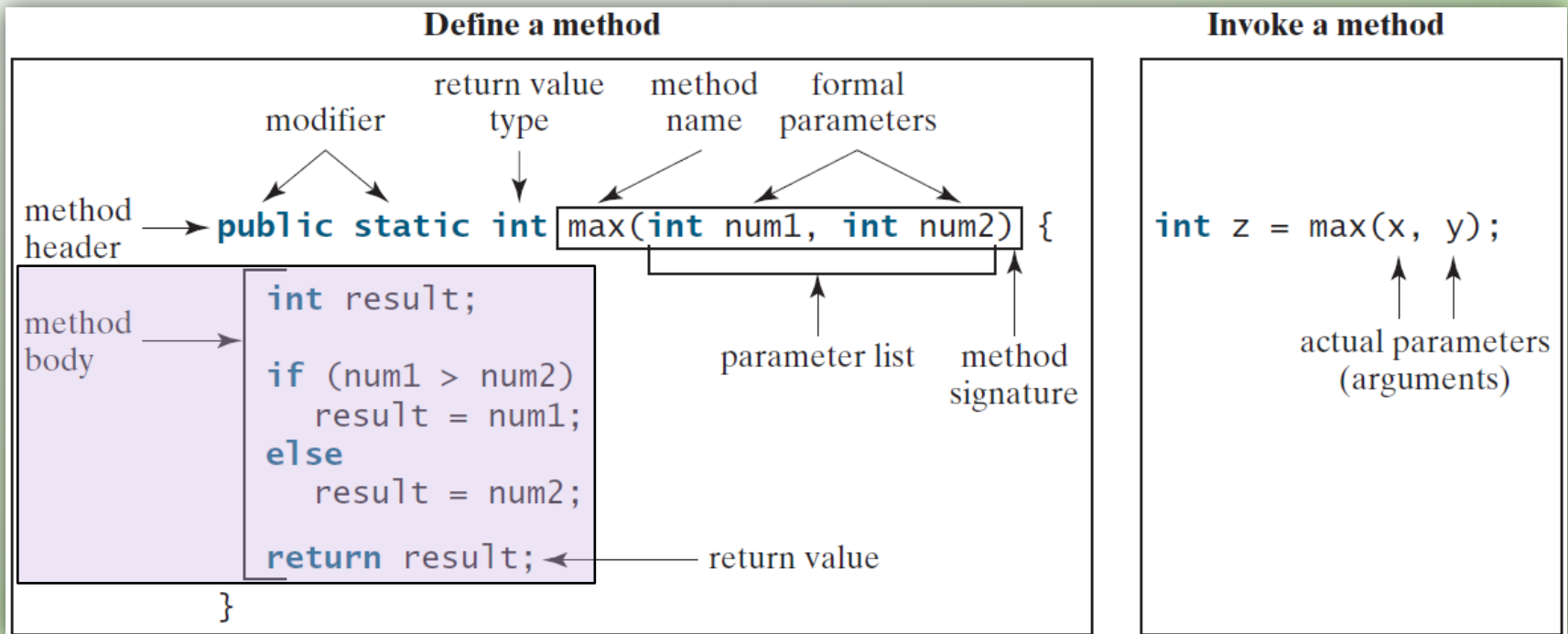




Defining a Method

♦ Method Body:

- Collection of statements that implement the method





Calling a Method

♦ Remember:

- A method is a collection of statements grouped together to perform an action
- So inside the method, you define the actions
 - ♦ You "do" everything that you want the method to "do"

♦ Question:

- How do we "start" the method? How do we run it?

♦ Answer:

- We *call* or *invoke* the method.



Calling a Method

♦ Two ways to call a method, depending on whether the method returns a value or not

1. If the method returns a value, the "call" is usually treated as a value:

♦ Example:

```
int larger = max(3, 4);
```

- Here, we "call" the method, `max(3, 4)`
- The maximum number, which is 4, will get returned
- We save that value (4) into the variable `larger`

♦ Example:

```
System.out.println(max(3, 4));
```

- Here, we directly print the result, which is 4



Calling a Method

- ♦ Two ways to call a method, depending on whether the method returns a value or not
 2. If the method returns **void**, the "call" to the method is a basic statement.
 - ♦ Example of the **println()** method:
System.out.println("This is a parameter.");
 - Here, we "call" the **println()** method
 - We send over the String, "This is a parameter"
 - That method receives the string and prints to output



Calling a Method

♦ Program Control

- When you run a program, the control of the program is in "**main**" (the main method)
 - ♦ This is called program control
- When you call a method from **main**, program control is transferred to the method you called
- **main** is basically waiting for the method to finish
 - ♦ Once the method finishes, program control returns to **main**
 - ♦ A called method returns control to the caller
 - when its return statement is executed, or
 - when its method-ending closing brace is reached



Program 2: TestMax

- ♦ Write a program that will call another method, **max**, to determine the maximum of two numbers. Method **max** should return the maximum value.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 2: TestMax

♦ Step 1: Problem-solving Phase

– Algorithm:

- ♦ In **main**, we just make two integers and give a value
 - Of course, we could ask the user for two numbers
 - Or we could generate two random numbers
 - These are easy things and are not the purpose of this example
- ♦ Next, we call the **max** method
- ♦ This means we need to write a **max** method!
 - **max** method should be easy
 - Just check which number is larger
 - Save the larger number into a variable
 - Finally, return that variable (the larger number)



Program 2: TestMax

♦ Step 2: Implementation

LISTING 6.1 TestMax.java

```
1  public class TestMax {
2      /** Main method */
3      public static void main(String[] args) {
4          int i = 5;
5          int j = 2;
6          int k = max(i, j);
7          System.out.println("The maximum of " + i +
8              " and " + j + " is " + k);
9      }
10
11     /** Return the max of two numbers */
12     public static int max(int num1, int num2) {
13         int result;
14
15         if (num1 > num2)
16             result = num1;
17         else
18             result = num2;
19
20         return result;
21     }
22 }
```



Program 2: TestMax

♦ Step 2: Implementation Details

- **main** method is from lines 3 – 9
- On line 6, **main** calls method **max**
- Method **max** is from lines 12 – 21
- On line 21, **result** is returned to **main**

LISTING 6.1 TestMax.java

```
1  public class TestMax {
2      /** Main method */
3      public static void main(String[] args) {
4          int i = 5;
5          int j = 2;
6          int k = max(i, j);
7          System.out.println("The maximum of " + i +
8              " and " + j + " is " + k);
9      }
10
11     /** Return the max of two numbers */
12     public static int max(int num1, int num2) {
13         int result;
14
15         if (num1 > num2)
16             result = num1;
17         else
18             result = num2;
19
20         return result;
21     }
22 }
```

Program 2: TestMax

♦ Run the Program:

The maximum of 5 and 2 is 5

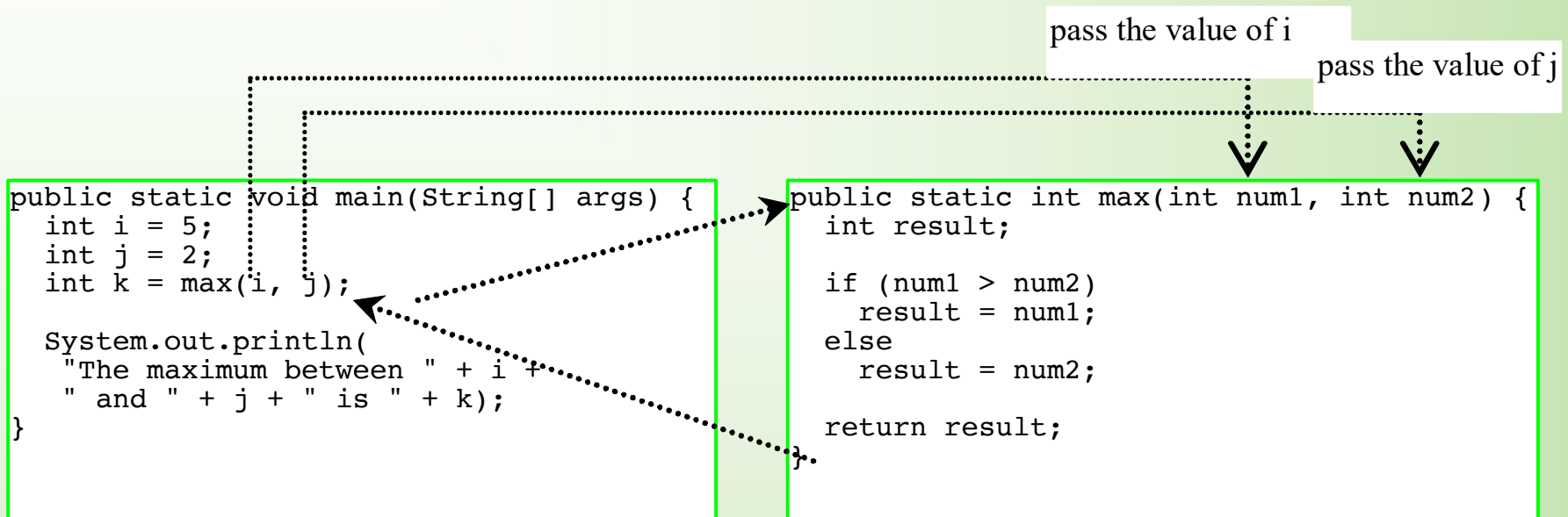


	line#	i	j	k	num1	num2	result
	4	5					
	5		2				
Invoking max {	12				5	2	
	13						undefined
	16						5
	6			5			



[Click here](#) to view and trace code

Calling Methods, cont.



Trace Method Invocation

i is now 5

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

j is now 2

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

invoke max(i, j)

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

After this line executes, program control go to method **max**.

Trace Method Invocation

invoke max(i, j)
Pass the value of i to num1
Pass the value of j to num2

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Program control is now at method **max**.

Trace Method Invocation

declare variable result

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

(num1 > num2) is true since num1 is 5 and num2 is 2

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

Trace Method Invocation

result is now 5

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

return result, which is 5

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

Now, the maximum value is returned.
Program control returns to **main**.

Trace Method Invocation

return max(i, j) and assign the
return value to k

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

Execute the print statement

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```



Caution

- ♦ A return statement is required for a value-returning method
- ♦ Consider the following method:
 - This method is 100% logically correct
 - However, it will give a compile error!
 - Why?

```
public static int sign(int n) {  
    if (n > 0)  
        return 1;  
    else if (n == 0)  
        return 0;  
    else if (n < 0)  
        return -1;  
}
```



Caution

♦ Why does this method give a compile error?

```
public static int sign(int n) {  
    if (n > 0)  
        return 1;  
    else if (n == 0)  
        return 0;  
    else if (n < 0)  
        return -1;  
}
```

- ♦ Java sees that this method must return an **int**
- ♦ Java makes sure that a value must always be returned
- ♦ Java does not allow the possibility that a value will not be returned
- ♦ If this possibility exists, Java will give a compile error



Caution

♦ Why does this method give a compile error?

```
public static int sign(int n) {  
    if (n > 0)  
        return 1;  
    else if (n == 0)  
        return 0;  
    else if (n < 0)  
        return -1;  
}
```

- ♦ Yes, this code is logically correct for us
 - Clearly, n can only be bigger than 0, equal, or less than zero
- ♦ However, Java sees the **if/elseif/elseif** block and Java expects that "maybe" there can be an "else" statement
- ♦ And if that else statement is true, Java has no guarantee that we will return inside that statement...therefore, an error

Caution

- ♦ Why does this method give a compile error?
 - In summary: Java thinks it is possible that this method will not return a value.
 - An easy solution is presented below:

```
public static int sign(int n) {  
    if (n > 0)  
        return 1;  
    else if (n == 0)  
        return 0;  
    else if (n < 0)  
        return -1;  
}
```

Should be →

```
public static int sign(int n) {  
    if (n > 0)  
        return 1;  
    else if (n == 0)  
        return 0;  
    else  
        return -1;  
}
```

- ♦ We simply remove the condition of the final else if statement, indicating that this is the final option



Calling a Method

♦ Activation Record:

- Each time a method is called, the system creates an activation record
- The activation record stores all parameters and variables for the method.
- The activation record is then placed in a specific area of memory known as a call stack.
 - ♦ aka "execution stack", "machine stack" or just "the stack"



Calling a Method

♦ Activation Record:

- When **methodA** calls **methodB**, for example, the activation record for **methodA** is kept intact
 - ♦ A new activation record for **methodB** is created for this new **methodB** that was just called
 - ♦ When **methodB** finishes its work and returns to the caller, which was **methodA**, the activation record of **methodB** is then removed from the stack of records
 - ♦ Why?
 - Because **methodB** is finished!
- Confused? Let us see an example...

Trace Call Stack

i is declared and initialized

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

i: 5

The main method
is invoked.

Trace Call Stack

j is declared and initialized

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

j: 2
i: 5

The main method
is invoked.

Trace Call Stack

Declare k

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:
j: 2
i: 5

The main method
is invoked.

Trace Call Stack

Invoke max(i, j)

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:
j: 2
i: 5

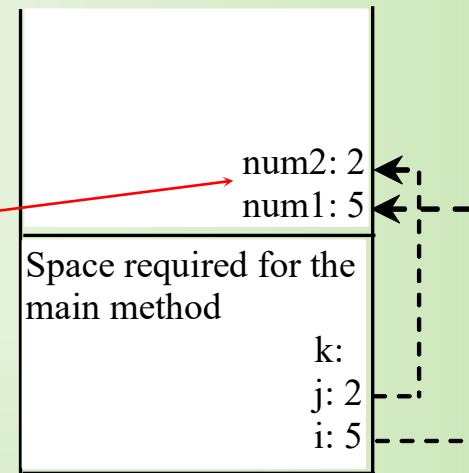
The main method
is invoked.

Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

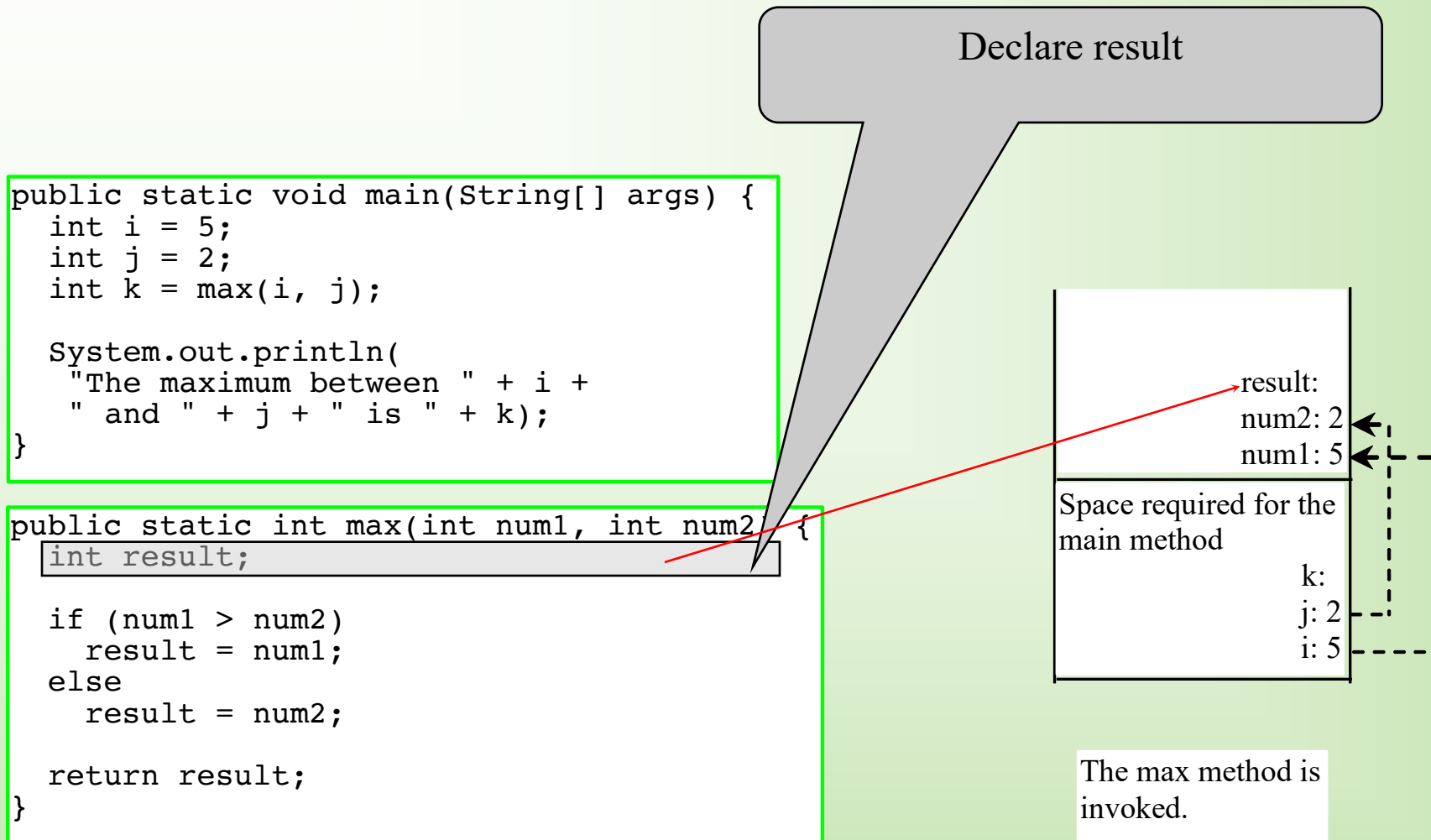
```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

pass the values of i and j to num1
and num2



The max method is
invoked.

Trace Call Stack



Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

(num1 > num2) is true

result:
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2)  
int result;  
  
if (num1 > num2)  
    result = num1;  
else  
    result = num2;  
  
return result;  
}
```

Assign num1 to result

Space required for the
max method

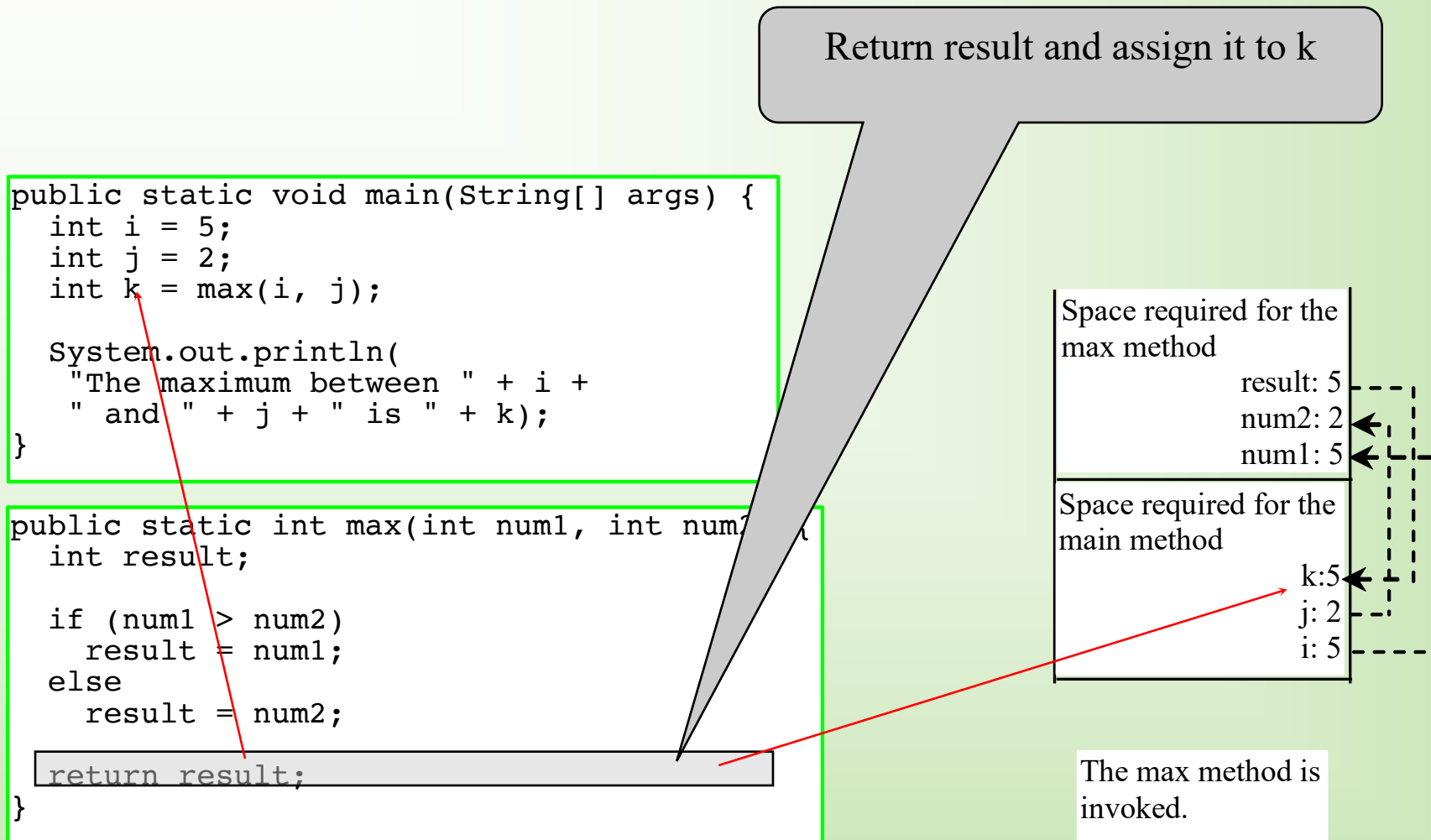
result: 5
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Trace Call Stack



Trace Call Stack

Execute print statement

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);
```

```
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

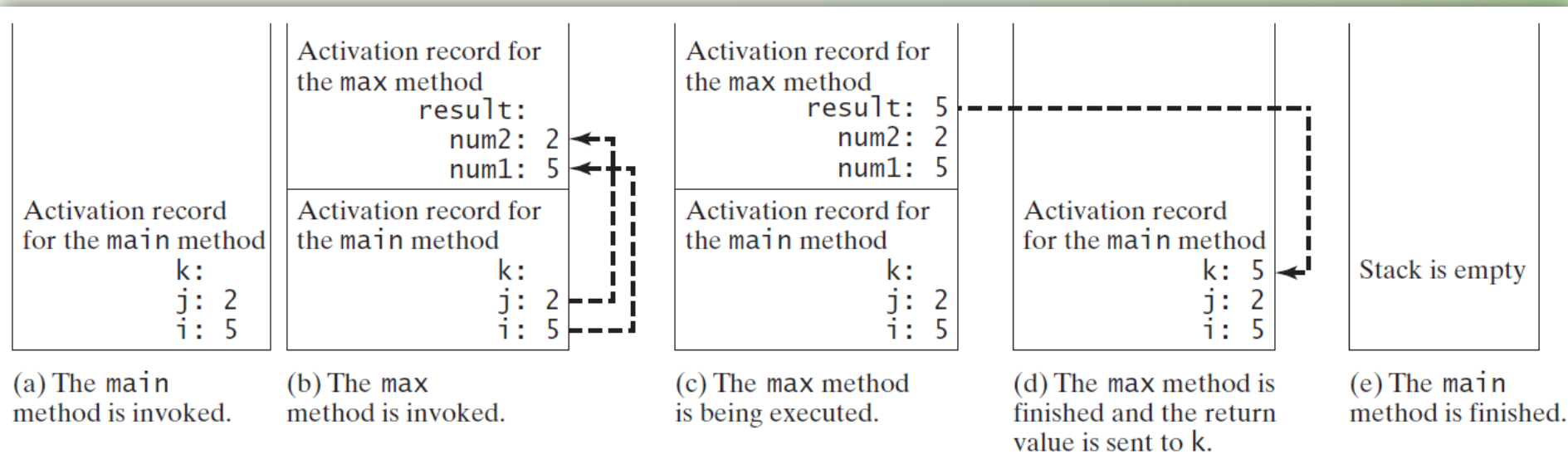
k:5
j:2
i:5

The main method
is invoked.

Calling a Method

♦ Activation Record and Call Stacks:

- While the last animated example was great, here is a single image that shows the full process:





void Method Example

- ♦ The previous example (**max** method) was a value-returning method
 - meaning, it returned a value (the **max**) to the caller
- ♦ Some methods do not return anything at all
- ♦ This type of method is called a **void** method
- ♦ The following program defines a method named **printGrade** and invokes (calls) it to print the grade based on a given score



Program 3: TestVoidMethod

LISTING 6.2 TestVoidMethod.java

```
1  public class TestVoidMethod {
2      public static void main(String[] args) {
3          System.out.print("The grade is ");
4          printGrade(78.5);
5
6          System.out.print("The grade is ");
7          printGrade(59.5);
8      }
9
10     public static void printGrade(double score) {
11         if (score >= 90.0) {
12             System.out.println('A');
13         }
14         else if (score >= 80.0) {
15             System.out.println('B');
16         }
17         else if (score >= 70.0) {
18             System.out.println('C');
19         }
20         else if (score >= 60.0) {
21             System.out.println('D');
22         }
23         else {
24             System.out.println('F');
25         }
26     }
27 }
```



Program 3: TestVoidMethod

♦ Run the Program:

```
The grade is C  
The grade is F
```



- The **printGrade** method is void
 - ♦ It does not return any value at all.
- A call to a **void** method must be a statement
- You can see it is invoked as a statement in line 4
- And like any Java statement, it ends with a semicolon

[Click here](#) to view and trace code



Program 4: TestReturnGradeMethod

- ♦ Redesign the **printGrade** method to return a value. The new method, **getGrade**, should return the letter grade as a **char** value.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 4: TestReturnGradeMethod

♦ Step 1: Problem-solving Phase

– Algorithm:

- ◆ Same algorithm as the previous example.
- ◆ We will call the method and we will send over the numeric grade as a parameter
- ◆ But instead of directly printing the value inside the method
- ◆ We will return the value as a char!



Program 4: TestReturnGradeMethod

♦ Step 2: Implementation

LISTING 6.3 TestReturnGradeMethod.java

```
1 public class TestReturnGradeMethod {
2     public static void main(String[] args) {
3         System.out.print("The grade is " + getGrade(78.5));
4         System.out.print("\nThe grade is " + getGrade(59.5));
5     }
6
7     public static char getGrade(double score) {
8         if (score >= 90.0)
9             return 'A';
10        else if (score >= 80.0)
11            return 'B';
12        else if (score >= 70.0)
13            return 'C';
14        else if (score >= 60.0)
15            return 'D';
16        else
17            return 'F';
18    }
19 }
```



Program 4: TestReturnGradeMethod

♦ Step 2: Implementation Details

- notice the return type for the **getGrade** method
 - ♦ **char**
- On line 3 and 4, we call the **getGrade** method
- The answer is computed inside the method, and then returned and printed

LISTING 6.3 TestReturnGradeMethod.java

```
1 public class TestReturnGradeMethod {
2     public static void main(String[] args) {
3         System.out.print("The grade is " + getGrade(78.5));
4         System.out.print("\nThe grade is " + getGrade(59.5));
5     }
6
7     public static char getGrade(double score) {
8         if (score >= 90.0)
9             return 'A';
10        else if (score >= 80.0)
11            return 'B';
12        else if (score >= 70.0)
13            return 'C';
14        else if (score >= 60.0)
15            return 'D';
16        else
17            return 'F';
18    }
19 }
```



Program 4: TestReturnGradeMethod

♦ Run the Program:

```
The grade is C  
The grade is F
```



[Click here](#) to view and trace code



void Method Example

♦ Note:

- the **return** statement is not needed for **void** method
- However, it can be useful for terminating the method and returning to the method's caller
- Syntax:
return;
- This is not too common
 - ♦ However, it is useful for exiting the normal flow of control in a void method
- Consider the following code, which uses a **void** method to terminate if the score is invalid



void Method Example

♦ void Method with return statement:

```
public static void printGrade(double score) {  
    if (score < 0 || score > 100) {  
        System.out.println("Invalid score");  
        return;  
    }  
    if (score >= 90.0) {  
        System.out.println('A');  
    }  
    else if (score >= 80.0) {  
        System.out.println('B');  
    }  
    else if (score >= 70.0) {  
        System.out.println('C');  
    }  
    else if (score >= 60.0) {  
        System.out.println('D');  
    }  
    else {  
        System.out.println('F');  
    }  
}
```



Check Point

- ♦ What are the benefits of using a method?
- ♦ Can a call to a value-returning method be a statement by itself?
- ♦ Can you have a return statement in a **void** method?



Passing Arguments by Values

- ♦ Power of a method is in its ability to work with parameters
- ♦ When you call a method, you need to provide the arguments (parameters)
 - And these parameters **MUST** be given in the same order as their respective parameters in the method signature
 - This is known as parameter order association



Passing Arguments by Values

♦ Example:

- We know the `println()` method works by taking one `String` parameter
 - ♦ Then it prints that `String` to the console
- What if we wanted to make a new `println()` method
- But this method will take two parameters
 - ♦ The `String` itself
 - ♦ And an integer, `n`, representing the number of times the method should print the `String`



Passing Arguments by Values

♦ Example:

```
public static void nPrintln(String message, int n) {  
    for (int i = 0; i < n; i++)  
        System.out.println(message);  
}
```

- What happens if we call `nPrintln("Hello", 3)`
 - ♦ Answer: we will print the word "Hello" three times
- The `String` parameter, "Hello" is passed into the formal parameter, `String message`
- And the value 3 is passed to the formal parameter, `n`



Passing Arguments by Values

♦ Example:

```
public static void nPrintln(String message, int n) {  
    for (int i = 0; i < n; i++)  
        System.out.println(message);  
}
```

- How would we print the specialty "Computer Science" 100 times?

nPrintn("Computer Science", 100);

- Note, the following statement would be wrong:

nPrintn(100, "Computer Science");

- Why?

♦ data types of the actual/formal parameters do not match



Passing Arguments by Values

♦ Caution:

- Arguments send from the caller MUST match the formal parameter with respect to:
 - ♦ The number of parameters
 - ♦ The order of parameters
 - ♦ The compatible type of parameters
 - Here, compatible simply means that you can pass an argument to a parameter without explicit casting
 - Such as passing an **int** to a **double**
 - This is allowed
 - But of course, we cannot pass a **double** to an **int**
 - Unless there was explicit casting



Passing Arguments by Values

♦ Remember:

- When you invoke a method with an argument, the value of the argument is passed/sent to the formal parameter inside the method
- This is referred to as *pass-by-value*
- Even if the argument is a variable, the value of the variable is passed to the formal parameter
- The variable, therefore, is not affected regardless of any changes made to the parameter inside the method
- Consider the following example...

Passing Arguments by Values

♦ Remember:

[Click here](#)
to view and
trace code

LISTING 6.4 Increment.java

```
1 public class Increment {  
2     public static void main(String[] args) {  
3         int x = 1;  
4         System.out.println("Before the call, x is " + x);  
5         increment(x);  
6         System.out.println("After the call, x is " + x);  
7     }  
8  
9     public static void increment(int n) {  
10        n++;  
11        System.out.println("n inside the method is " + n);  
12    }  
13 }
```



Before the call, x is 1
n inside the method is 2
After the call, x is 1



Passing Arguments by Values

♦ And another example:

- We have seen that swapping the values of two variables is a common practice
 - ♦ So let us write a method to do this!
- Consider the following program that creates a method for swapping the values of two variables
 - ♦ So we will make a custom **swap** method
- We invoke the **swap** method by passing 2 arguments
- Interesting:
 - ♦ The values of the arguments are NOT changed after the method is invoked



Passing Arguments by Values

♦ And another example:

LISTING 6.5 TestPassByValue.java

```
1  public class TestPassByValue {
2      /** Main method */
3      public static void main(String[] args) {
4          // Declare and initialize variables
5          int num1 = 1;
6          int num2 = 2;
7
8          System.out.println("Before invoking the swap method, num1 is " +
9              num1 + " and num2 is " + num2);
10
11         // Invoke the swap method to attempt to swap two variables
12         swap(num1, num2);
13
14         System.out.println("After invoking the swap method, num1 is " +
15             num1 + " and num2 is " + num2);
16     }
17 }
```



Passing Arguments by Values

♦ And another example:

```
18  /** Swap two variables */
19  public static void swap(int n1, int n2) {
20      System.out.println("\tInside the swap method");
21      System.out.println("\t\tBefore swapping, n1 is " + n1
22          + " and n2 is " + n2);
23
24      // Swap n1 with n2
25      int temp = n1;
26      n1 = n2;
27      n2 = temp;
28
29      System.out.println("\t\tAfter swapping, n1 is " + n1
30          + " and n2 is " + n2);
31  }
32 }
```

Passing Arguments by Values

♦ And another example:

– Output:

```
Before invoking the swap method, num1 is 1 and num2 is 2
  Inside the swap method
    Before swapping, n1 is 1 and n2 is 2
    After swapping, n1 is 2 and n2 is 1
  After invoking the swap method, num1 is 1 and num2 is 2
```



[Click here](#) to view and trace code



Passing Arguments by Values

♦ **swap** Method Explanation:

- Before **swap** method is invoked (line 12) **num1** is **1** and **num2** is **2**
- After the **swap** method is invoked, **num1** is still **1** and **num2** is still **2**.
 - ♦ Their values have NOT been swapped!
- Why?
 - ♦ Values in a method have their own memory locations
 - ♦ Therefore, any changes made inside the method do NOT affect the original variables outside the method



Passing Arguments by Values

♦ **swap** Method Explanation:

– Common question:

♦ "In this example, the variables names were different when we look at the formal and actual parameters. Would it make a difference if the names were the same?

– Answer:

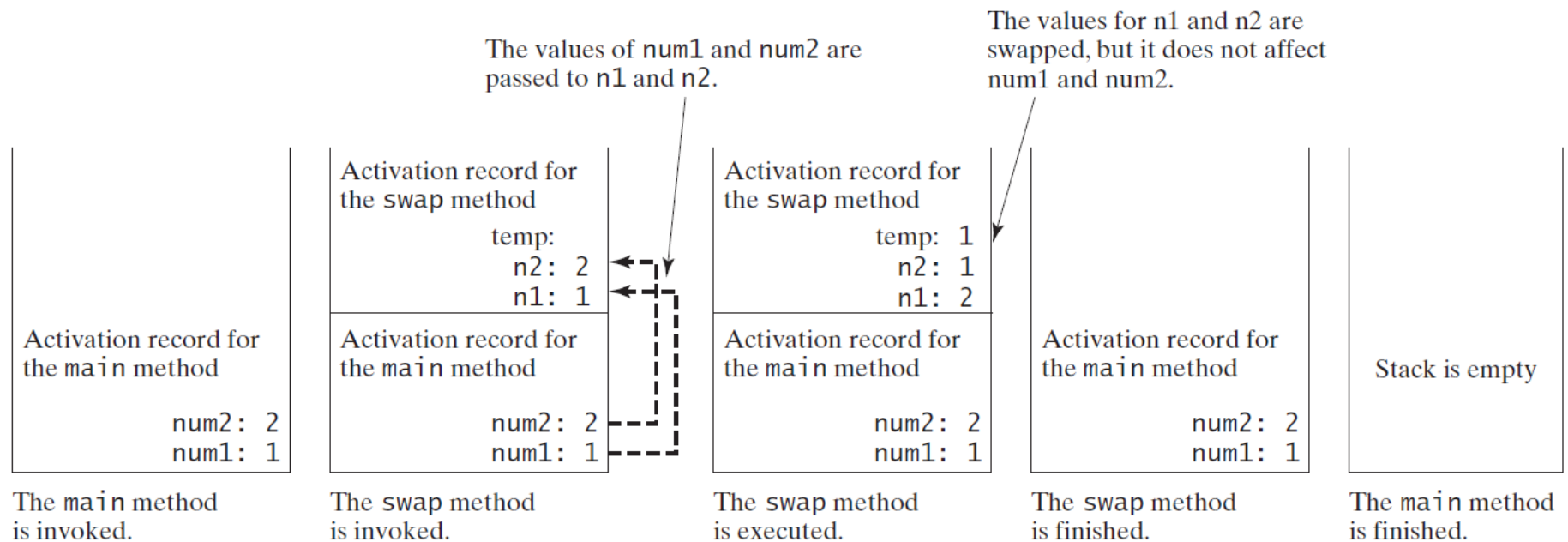
♦ No. Nothing will change.

♦ The parameter is a variable in the method with its own memory space.

– Consider the following Activation Record for **swap**

Passing Arguments by Values

♦ Activation Record for **swap**:





Check Point

♦ What is the output of the following programs:

```
public class Test {  
    public static void main(String[] args) {  
        int max = 0;  
        max(1, 2, max);  
        System.out.println(max);  
    }  
  
    public static void max(  
        int value1, int value2, int max) {  
        if (value1 > value2)  
            max = value1;  
        else  
            max = value2;  
    }  
}
```

(a)

```
public class Test {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 6) {  
            method1(i, 2);  
            i++;  
        }  
    }  
  
    public static void method1(  
        int i, int num) {  
        for (int j = 1; j <= i; j++) {  
            System.out.print(num + " ");  
            num *= 2;  
        }  
  
        System.out.println();  
    }  
}
```

(b)



Check Point

♦ What is the output of the following programs:

```
public class Test {  
    public static void main(String[] args) {  
        // Initialize times  
        int times = 3;  
        System.out.println("Before the call,"  
            + " variable times is " + times);  
  
        // Invoke nPrintln and display times  
        nPrintln("Welcome to Java!", times);  
        System.out.println("After the call,"  
            + " variable times is " + times);  
    }  
  
    // Print the message n times  
    public static void nPrintln(  
        String message, int n) {  
        while (n > 0) {  
            System.out.println("n = " + n);  
            System.out.println(message);  
            n--;  
        }  
    }  
}
```

(c)

```
public class Test {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i <= 4) {  
            method1(i);  
            i++;  
        }  
  
        System.out.println("i is " + i);  
    }  
  
    public static void method1(int i) {  
        do {  
            if (i % 3 != 0)  
                System.out.print(i + " ");  
            i--;  
        }  
        while (i >= 1);  
  
        System.out.println();  
    }  
}
```

(d)



Modularizing Code

- ♦ What is the idea of modularizing code?
 - To answer this, let us ask another question:
What is a module?
 - Answer: a sub-group of a larger entity
 - For example, you have a Chapter in your book, and then inside the chapter, maybe you have 8 modules
 - These are small, independent sections of the Chapter
- ♦ Imagine if the chapter did not have modules, and you were told to “modularize the chapter”.
 - This means, divide the chapter into modules!



Modularizing Code

- ♦ What is the idea of modularizing code?
 - This same idea applies to code
 - New programmers often write long un-modularized code, which is very difficult to read
 - So we tell them: modularize the code!
 - This makes the code easier:
 - ♦ To maintain
 - ♦ To read
 - ♦ To debug
 - ♦ and a best of all, it makes the code reusable!



Modularizing Code

♦ Use of methods:

- We already learned that methods can be used to reduce redundant code and they facilitate reuse of code
- Methods are also used to modularize code and to help improve the overall quality of the program

♦ GCD (modularized)

- In Chapter 5, we wrote a program to find the GCD of two integers
- We can now re-write the program in a modularized fashion by using a method to compute the GCD



Modularizing Code (GCD)

♦ GCD (modularized)

LISTING 6.6 GreatestCommonDivisorMethod.java

```
1  import java.util.Scanner;
2
3  public class GreatestCommonDivisorMethod {
4      /** Main method */
5      public static void main(String[] args) {
6          // Create a Scanner
7          Scanner input = new Scanner(System.in);
8
9          // Prompt the user to enter two integers
10         System.out.print("Enter first integer: ");
11         int n1 = input.nextInt();
12         System.out.print("Enter second integer: ");
13         int n2 = input.nextInt();
14     }
```



Modularizing Code (GCD)

♦ GCD (modularized)

[Click here](#) to view and trace code

```
15     System.out.println("The greatest common divisor for " + n1 +
16         " and " + n2 + " is " + gcd(n1, n2));
17 }
18
19 /** Return the gcd of two integers */
20 public static int gcd(int n1, int n2) {
21     int gcd = 1; // Initial gcd is 1
22     int k = 2; // Possible gcd
23
24     while (k <= n1 && k <= n2) {
25         if (n1 % k == 0 && n2 % k == 0)
26             gcd = k; // Update gcd
27         k++;
28     }
29
30     return gcd; // Return gcd
31 }
32 }
```



Modularizing Code (GCD)

♦ Run the Program:

[Click here](#) to view and trace code



Enter first integer: 45

Enter second integer: 75

The greatest common divisor for 45 and 75 is 15

- We encapsulated the GCD code inside a method
- Benefits:
 - ♦ Isolates the GCD computation from the rest of the program
 - ♦ Possible errors on GCD are limited to GCD method
 - makes debugging easier
 - ♦ GCD method can now be reused by other programs



Program 5: PrimeNumberMethod

- ♦ Write a modularized program, which should print the first 50 prime numbers, with ten numbers printed per line
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 5: PrimeNumberMethod

♦ Step 1: Problem-solving Phase

– Algorithm:

- ♦ We really have two things to consider for this program:
 1. We need to determine if a number is prime
 2. We need to print 10 prime numbers per line
- ♦ And we can do both of these steps with methods
- ♦ We can make a method, `isPrime`, to determine prime
- ♦ And we can make another method that is used specifically to print the numbers, `printPrimeNumbers`



Program 5: PrimeNumberMethod

♦ Step 1: Problem-solving Phase

– Algorithm:

♦ In Chapter 5, we showed how to determine if a # is prime:

```
// Use a boolean variable isPrime and set initially to true
boolean isPrime = true;

for (int divisor = 2; divisor <= number/2; divisor++) {
    // IF divisor evenly divides into number, number is NOT prime
    if (number % divisor == 0) {
        isPrime = false;
        break; // Exit/break out of loop!
    }
}
```



Program 5: PrimeNumberMethod

♦ Step 1: Problem-solving Phase

– Algorithm:

♦ Method printPrimeNumbers

– We can keep a final variable: `NUMBER_OF_PRIMES_PER_LINE`

♦ We keep a counter to count the number of primes found

♦ We use the same while loop from Chapter 5

```
while (count < numberOfPrimes)
```

♦ We start with the number 2, and check if it is prime

♦ If so, we increment count

♦ Of course, we increment the tested value

♦ And we continue until we find the desired # of primes



Program 5: PrimeNumberMethod

♦ Step 2: Implementation

LISTING 6.7 PrimeNumberMethod.java

```
1 public class PrimeNumberMethod {
2     public static void main(String[] args) {
3         System.out.println("The first 50 prime numbers are \n");
4         printPrimeNumbers(50);
5     }
6
7     public static void printPrimeNumbers(int numberOfPrimes) {
8         final int NUMBER_OF_PRIMES_PER_LINE = 10; // Display 10 per line
9         int count = 0; // Count the number of prime numbers
10        int number = 2; // A number to be tested for primeness
11
12        // Repeatedly find prime numbers
13        while (count < numberOfPrimes) {
14            // Print the prime number and increase the count
15            if (isPrime(number)) {
16                count++; // Increase the count
17            }
18        }
19    }
20}
```



Program 5: PrimeNumberMethod

♦ Step 2: Implementation

```
18         if (count % NUMBER_OF_PRIMES_PER_LINE == 0) {
19             // Print the number and advance to the new line
20             System.out.printf("%-5s\n", number);
21         }
22         else
23             System.out.printf("%-5s", number);
24     }
25
26     // Check whether the next number is prime
27     number++;
28 }
29 }
30
31 /** Check whether number is prime */
32 public static boolean isPrime(int number) {           isPrime method
33     for (int divisor = 2; divisor <= number / 2; divisor++) {
34         if (number % divisor == 0) { // If true, number is not prime
35             return false; // Number is not a prime
36         }
37     }
38
39     return true; // Number is prime
40 }
41 }
```

Program 5: PrimeNumberMethod

♦ Run the Program:

The first 50 prime numbers are

2	3	5	7	11	13	17	19	23	29
31	37	41	43	47	53	59	61	67	71
73	79	83	89	97	101	103	107	109	113
127	131	137	139	149	151	157	163	167	173
179	181	191	193	197	199	211	223	227	229



[Click here](#) to view and trace code



Overloading Methods

- ♦ What is method overloading?
 - Method overloading is when you have multiple methods with the SAME method name
 - However, the method signature is different
 - This means that the number of parameters will be different or the data types of the parameters will be different



Overloading Methods

♦ Example **max** method:

- Previously, we made a **max** method to find the max value of two integers
- What if the two values were doubles?
- Solution:
 - ♦ Create another method with the same name
 - ♦ The only difference is that the data type of the parameters will be double
- So now your program has two **max** methods
 - ♦ One that works with integers and one that works for doubles



Overloading Methods

♦ Example **max** method:

- So now if you call the max method with two integers, the specific method that expects two integers will be invoked
- And if you call the max method with two doubles, the specific method that expects two doubles will be invoked
- This is referred to as method overloading
- The Java compiler determines which method to use based on the method signature



Program 6: TestMethodOverloading

- ♦ Write a program that finds the max of two integers, the max of two doubles, and the max of three doubles
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 6: TestMethodOverloading

♦ Step 1: Problem-solving Phase

– Algorithm:

- ♦ We need three methods
- ♦ And all of them will be called max!
- ♦ One will compute max of two integers
- ♦ One will compute max of two doubles
- ♦ One will compute max of three doubles



Program 6: TestMethodOverloading

♦ Step 2: Implementation

LISTING 6.9 TestMethodOverloading.java

```
1  public class TestMethodOverloading {
2      /** Main method */
3      public static void main(String[] args) {
4          // Invoke the max method with int parameters
5          System.out.println("The maximum of 3 and 4 is "
6              + max(3, 4));
7
8          // Invoke the max method with the double parameters
9          System.out.println("The maximum of 3.0 and 5.4 is "
10             + max(3.0, 5.4));
11
12         // Invoke the max method with three double parameters
13         System.out.println("The maximum of 3.0, 5.4, and 10.14 is "
14             + max(3.0, 5.4, 10.14));
15     }
16 }
```



Program 6: TestMethodOverloading

♦ Step 2: Implementation

```
17  /** Return the max of two int values */
18  public static int max(int num1, int num2) {
19      if (num1 > num2)
20          return num1;
21      else
22          return num2;
23  }
24
25  /** Find the max of two double values */
26  public static double max(double num1, double num2) {
27      if (num1 > num2)
28          return num1;
29      else
30          return num2;
31  }
32
33  /** Return the max of three double values */
34  public static double max(double num1, double num2, double num3) {
35      return max(max(num1, num2), num3);
36  }
37  }
```



Program 6: TestMethodOverloading

♦ Run the Program:



```
The maximum of 3 and 4 is 4  
The maximum of 3.0 and 5.4 is 5.4  
The maximum of 3.0, 5.4, and 10.14 is 10.14
```

[Click here](#) to view and trace code



Ambiguous Invocation

- ♦ What is Ambiguous Invocation?
 - Sometimes there are two (or more) possible methods that match the invocation (the method call)
 - The result: the compiler cannot determine which method should be used
 - This is referred to as ambiguous invocation
 - ♦ ambiguous means “not clear”
 - Ambiguous invocations will cause a compile error



Ambiguous Invocation

♦ Example:

```
public class AmbiguousOverloading {
    public static void main(String[] args) {
        System.out.println(max(1, 2));
    }
    public static double max(int num1, double num2) {
        if (num1 > num2)
            return num1;
        else
            return num2;
    }
    public static double max(double num1, int num2) {
        if (num1 > num2)
            return num1;
        else
            return num2;
    }
}
```



Ambiguous Invocation

♦ Example:

- So in this Class, we have two **max** methods
- Both have two parameters:
 - ♦ The 1st has: **int** and **double**
 - ♦ The 2nd method has: **double** and **int**
- Neither method is better than the other
- In summary: the compiler cannot choose a “best” option
- Result: compiler error



Check Point

♦ What is wrong in the following program:

```
public class Test {  
    public static void someMethod(int x) {  
  
    }  
  
    public static int someMethod(int y) {  
        return y;  
    }  
}
```

- Signatures **MUST** be different
- Here, they are the same!



Check Point

♦ Given two method definitions:

```
public static double m(double x, double y)
```

```
public static double m(int x, double y)
```

tell which of the two methods is invoked for:

- a) `double z = m(4, 5);`
- b) `double z = m(4, 5.4);`
- c) `double z = m(4.5, 5.4);`



Scope of Local Variables

◆ Reminder from Chapter 2:

- The scope of a variable is the part of the program where the variable is understood
 - ◆ While we discussed “scope” previously, we now discuss it in more detail!
 - ◆ A variable declared inside a method is referred to as a local variable
 - ◆ The scope of a local variable begins from the declaration and continues until the end of the block that contains the variable
 - ◆ A local variable must be declared and assigned a value before it can be used



Scope of Local Variables

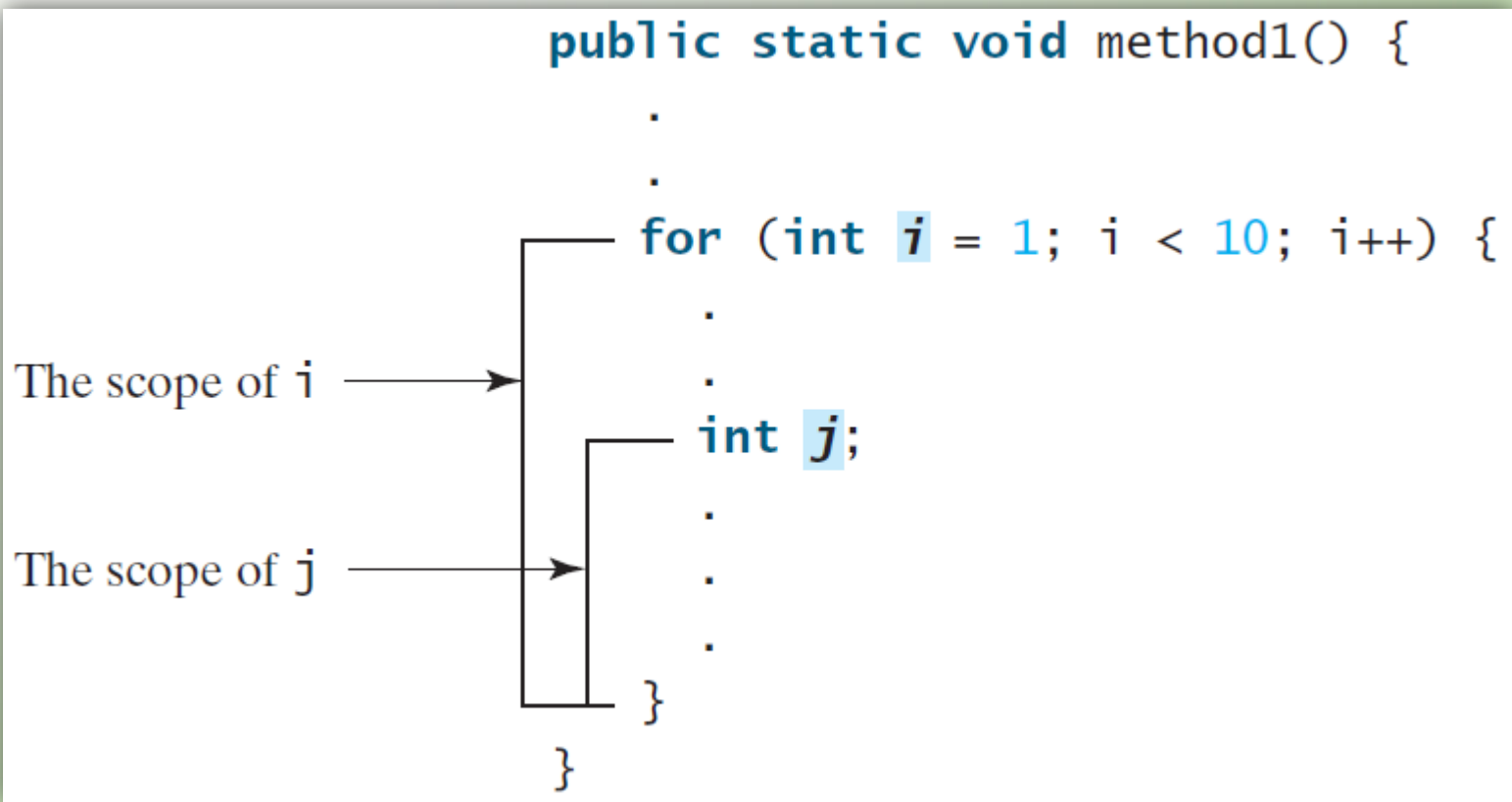
♦ Scope details:

- A parameter is a local variable
- And a parameter is “declared” inside the method signature
 - ♦ This means the scope of parameters are for the entire method!
- A variable declared in the initial-action part of a **for** loop has its scope in the entire loop
- However, a variable declared inside a **for**-loop body has its scope limited inside the loop body
 - ♦ from the declaration to the end of the **for**-loop block



Scope of Local Variables

♦ Scope details:





Scope of Local Variables

♦ Scope details:

- You can declare a local variable with the same name in different blocks in a method.
 - ♦ So even though the name is the same, you can do this with the condition IF it is in different blocks in the method
- You cannot declare a local variable twice within the same block or in nested blocks
- Consider the following picture...

Scope of Local Variables

◆ Scope details:

It is fine to declare `i` in two nonnested blocks.

```
public static void method1() {  
    int x = 1;  
    int y = 1;  
  
    for (int i = 1; i < 10; i++) {  
        x += i;  
    }  
  
    for (int i = 1; i < 10; i++) {  
        y += i;  
    }  
}
```

It is wrong to declare `i` in two nested blocks.

```
public static void method2() {  
    int i = 1;  
    int sum = 0;  
  
    for (int i = 1; i < 10; i++) {  
        sum += i;  
    }  
}
```



Program 7: RandomCharacter

- ♦ Write a class, which has methods to generate a random character, a random lowercase letter, a random uppercase letter, and a random digit character. Then write a program to print 175 random lowercase letters.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 7: RandomCharacter

♦ Step 1: Problem-solving Phase

– Reminder:

- ♦ A character is coded using an integer
- ♦ So generating a random character really means generating a random integer!
- ♦ Remember: every possible character has a Unicode value between 0 and FFFF in hexadecimal (65535 in decimal)
- ♦ So generating a random character really means generating a random decimal value between 0 and 65535 inclusive

```
(int) (Math.random() * (65535 + 1))
```



Program 7: RandomCharacter

♦ Step 1: Problem-solving Phase

– Details:

- ♦ So how do we generate a random lowercase letter?
- ♦ Well, you can either remember the exact decimal values for each character (hard to remember)
- ♦ OR you can just cast the character values as ints

`(int) 'a'`

– For example, the line above will give us the integer value of 'a'

- ♦ So we want a random integer between `(int) 'a'` and `(int) 'z'`

- ♦ Solution:

```
(int)((int)'a' + Math.random() * ((int)'z' - (int)'a' + 1))
```



Program 7: RandomCharacter

♦ Step 1: Problem-solving Phase

– Details:

- ♦ It is actually even easier than this though!
- ♦ All numeric operators can be applied to char operands
- ♦ The char operand will be implicitly cast to an int if the other operand is an int or a char
- ♦ Therefore, the previous expression can be simplified to:

`'a' + Math.random() * ('z' - 'a' + 1)`

- ♦ And a random lowercase letter is:

`(char)('a' + Math.random() * ('z' - 'a' + 1))`



Program 7: RandomCharacter

♦ Step 1: Problem-solving Phase

– Details:

- ◆ This allows us to realize a very simply but useful discovery:
- ◆ A random character between any two characters, ch1 and ch2 (with $ch1 < ch2$) can be generated as follows:

```
(char)(ch1 + Math.random() * (ch2 - ch1 + 1))
```

- ◆ So now we can make a generic method, which will have two characters as parameters
- ◆ And this generic method will generate the random character between those two character parameters
- ◆ We now make a class with five overloaded methods to get a certain type of character randomly



Program 7: RandomCharacter

♦ Step 2: Implementation

LISTING 6.10 RandomCharacter.java

```
1  public class RandomCharacter {
2      /** Generate a random character between ch1 and ch2 */
3      public static char getRandomCharacter(char ch1, char ch2) {
4          return (char)(ch1 + Math.random() * (ch2 - ch1 + 1));
5      }
6
7      /** Generate a random lowercase letter */
8      public static char getRandomLowerCaseLetter() {
9          return getRandomCharacter('a', 'z');
10     }
11
12     /** Generate a random uppercase letter */
13     public static char getRandomUpperCaseLetter() {
14         return getRandomCharacter('A', 'Z');
15     }
16 }
```



Program 7: RandomCharacter

♦ Step 2: Implementation

```
17  /** Generate a random digit character */
18  public static char getRandomDigitCharacter() {
19      return getRandomCharacter('0', '9');
20  }
21
22  /** Generate a random character */
23  public static char getRandomCharacter() {
24      return getRandomCharacter('\u0000', '\uFFFF');
25  }
26 }
```



Program 7: RandomCharacter

♦ Step 1: Problem-solving Phase

– Algorithm:

- ♦ So now we need to solve the original problem:
Print 175 random lowercase letters
 - Only 25 letters per line
- ♦ This is easy!
- ♦ We will have a loop, and inside the loop, we will simply call our method each time, of course with 'a' and 'z' as parameters



Program 7: RandomCharacter

♦ Step 2: Implementation

LISTING 6.11 TestRandomCharacter.java

```
1  public class TestRandomCharacter {
2      /** Main method */
3      public static void main(String[] args) {
4          final int NUMBER_OF_CHARS = 175;
5          final int CHARS_PER_LINE = 25;
6
7          // Print random characters between 'a' and 'z', 25 chars per line
8          for (int i = 0; i < NUMBER_OF_CHARS; i++) {
9              char ch = RandomCharacter.getRandomLowerCaseLetter();
10             if ((i + 1) % CHARS_PER_LINE == 0)
11                 System.out.println(ch);
12             else
13                 System.out.print(ch);
14         }
15     }
16 }
```

Program 7: RandomCharacter

♦ Run the Program:

```
gmjsohezfkgtazqgmswfc1rao  
pnrunulnwmaztlfjedmpchcif  
1alqdgivxkxpbzulrmqmbhikr  
1bnrjlsopfxahssqhwuuljvbe  
xbhdotzhpehbqmuwsfktwsoli  
cbuwkzgxpmtzihgatdslvbwbz  
bfesoklwbhnooygiigzdxuqni
```



[Click here](#) to view and trace code



Method Abstraction

- ♦ Main idea for developing software!
 - To develop quality software, programmers must fully understand and be comfortable with the idea of method abstraction
- ♦ What is method abstraction?
 - We separate the implementation of a method from the actual use of the method
 - The client/customer can use a method without knowing how to actually code it
 - The details of the method are hidden from the client



Method Abstraction

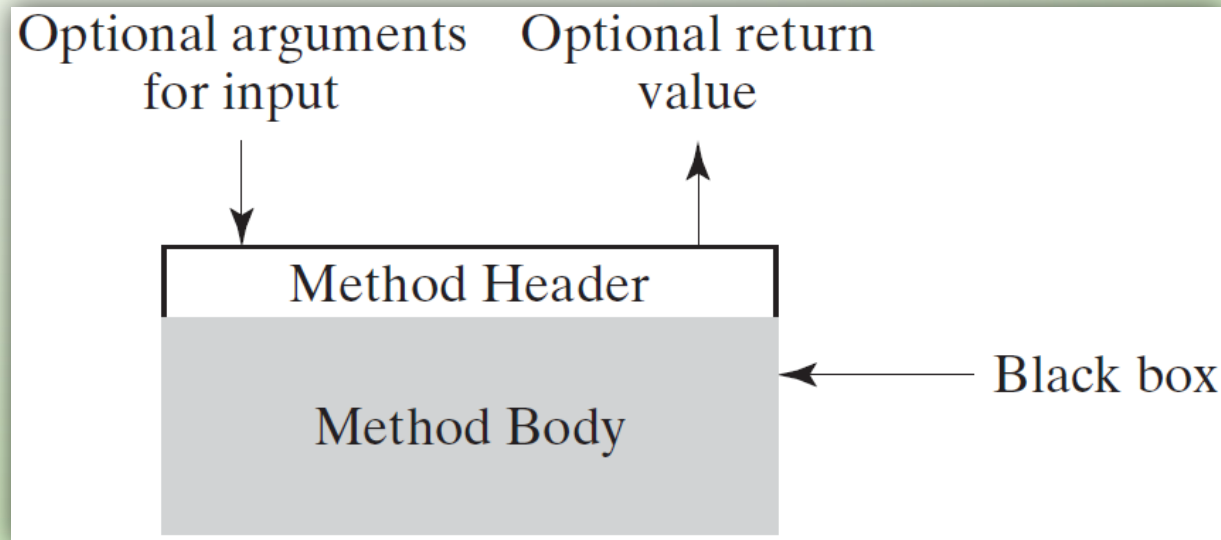
♦ Information Hiding (Encapsulation)

- Again, the details of the implementation are encapsulated inside the method
- And they are hidden from the client
- This is called Information Hiding or encapsulation
- The client has access to the method header
 - ♦ They can call the method with certain parameters
 - ♦ And they hope to get a return value from the method
- But what is inside the method is hidden from them
- In fact, they don't care...they just want it to work!

Method Abstraction

♦ Information Hiding (Encapsulation)

- So think of the method as a “BLACK BOX” that contains the implementation...but it is hidden





Stepwise Refinement

- ◆ Method Abstraction helps makes programs easier
 - because the implementation of a specific idea is removed from the main body of the program
 - So the program is easier to read and understand
- ◆ This idea is part of Stepwise Refinement
- ◆ What is stepwise refinement?
 - The idea of solving a larger problem/program in smaller steps
 - Certainly, solving something small is easier than solving something larger



Stepwise Refinement

- ♦ Method Abstraction helps makes programs easier
 - because the implementation of a specific idea is removed from the main body of the program
 - So the program is easier to read and understand
- ♦ This idea is part of Stepwise Refinement
- ♦ What is stepwise refinement?
 - The idea of solving a larger problem/program in smaller steps
 - Certainly, solving something small is easier than solving something larger



Program 8: PrintCalendar

- ♦ Write a program that prompts the user to enter the calendar year and month and then displays the exact calendar for that input.

- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 8: PrintCalendar

♦ Step 1: Problem-solving Phase

– Expected output:



Enter full year (e.g., 2012): 2012

Enter month as number between 1 and 12: 3

March 2012

Sun	Mon	Tue	Wed	Thu	Fri	Sat
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	



Program 8: PrintCalendar

♦ Step 1: Problem-solving Phase

– Requirements:

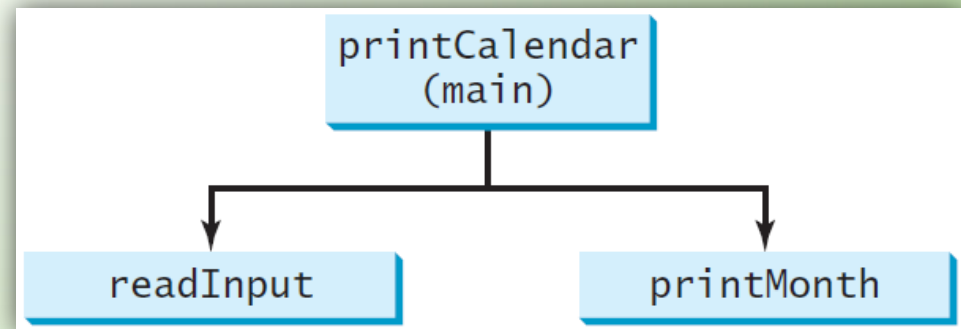
- ◆ First requirement: do NOT START CODING!!!
 - ◆ New programmers want to start code right away
 - ◆ And they also care about the DETAILS of the program
 - ◆ Yes, details are important...but not at the beginning
- The main requirement is to truly understand what the programming is asking of you
- So for this problem, let us use method abstraction to isolate the details from the actual program design

Program 8: PrintCalendar

♦ Step 1: Problem-solving Phase

– Problem Components:

- ♦ We can start by breaking the program into two main components:
 - Get input from user
 - Print the calendar



- ♦ Clearly, getting input from the user is easy and can be left for later discussion
- ♦ The main work is in printing the calendar



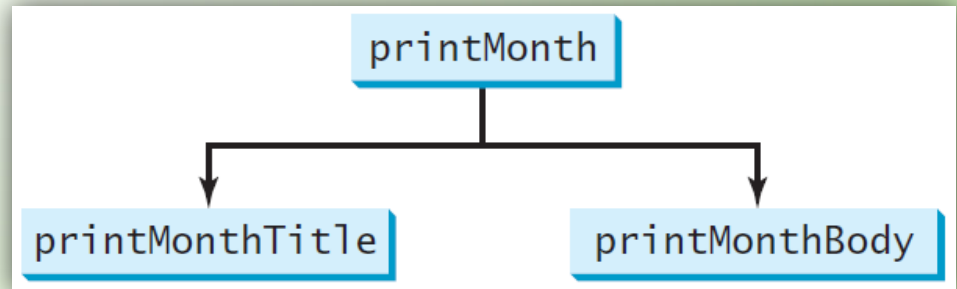
Program 8: PrintCalendar

♦ Step 1: Problem-solving Phase

– Problem Components:

- ♦ And printing the calendar can also be broken down into two components:

- Print the month title
- Print the month body



- ♦ Printing the month title is easy

- It consists of three lines, month and year, a long dashed line, and then the names of the week
- The only “calculation” here is determining the name of the month

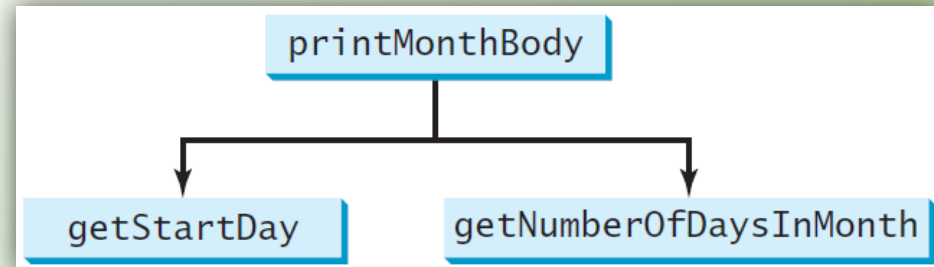


Program 8: PrintCalendar

♦ Step 1: Problem-solving Phase

– Problem Components:

- ♦ Printing the month body will take some thought
- ♦ There are two main things we must compute
 - Starting day of month
 - # of days in month



- ♦ So how can you get the starting day of the month?
 - This problem on its own can be complicated and requires its own thought and strategy



Program 8: PrintCalendar

♦ Step 1: Problem-solving Phase

- So how can you get the starting day of the month?
 - ♦ Assume we know that the start day for January 1, 1800 was a Wednesday

START_DAY_FOR_JAN_1_1800 = 3

- ♦ You could compute the total number of days between January 1, 1800 and the first date of the calendar month
- ♦ The start day of the calendar month is:
 $(\text{totalNumberOfDays} + \text{START_DAY_FOR_JAN_1_1800}) \% 7$
- ♦ Summary: the problem of getting the starting day can be further broken down into the problem of getting the total number of days since January 1, 1800



Program 8: PrintCalendar

♦ Step 1: Problem-solving Phase

- Okay. So how can we get the total number of days?
 - ♦ Simple, each year is 365 days.
 - ♦ And then for the last year, you must count the number of days before that specific month
 - ♦ This means you need to save the number of days in each month
 - And you can write a separate method for this
 - ♦ But wait! There is something else to consider!
 - ♦ LEAP YEAR!
 - ♦ So you must also test for a leap year



Program 8: PrintCalendar

♦ Step 1: Problem-solving Phase

– Method **getNumberOfDaysInMonth()**:

```
public static int getNumberOfDaysInMonth(int year, int month) {  
    if (month == 1 || month == 3 || month == 5 || month == 7 ||  
        month == 8 || month == 10 || month == 12)  
        return 31;  
  
    if (month == 4 || month == 6 || month == 9 || month == 11)  
        return 30;  
  
    if (month == 2)  
        return isLeapYear(year) ? 29 : 28;  
    return 0; // If month is incorrect  
}  
  
/** Determine if it is a leap year */  
public static boolean isLeapYear(int year) {  
    return year % 400 == 0 || (year % 4 == 0 && year % 100 != 0);  
}
```



Program 8: PrintCalendar

♦ Step 1: Problem-solving Phase

- So you can see that many components are needed to solve this problem
- You cannot just start coding immediately
- Instead, you must identify, step-by-step, or component-by-component, what is needed for your program
- What we just did was called the “Top-Down Approach”
- The design diagram is shown on the next pages

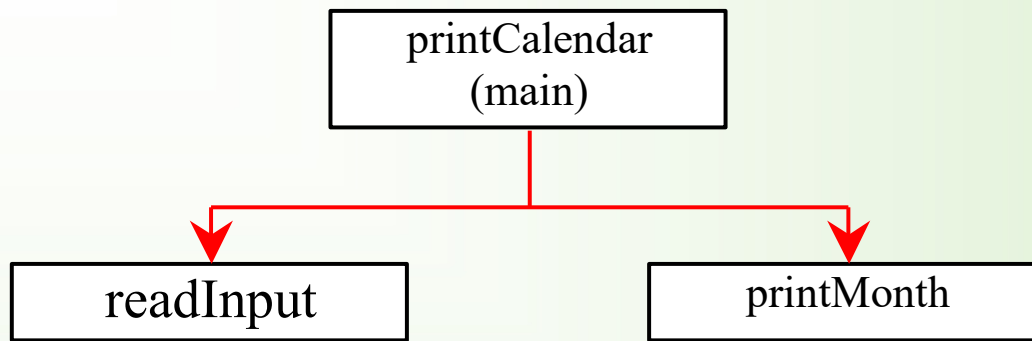


Design Diagram

printCalendar
(main)

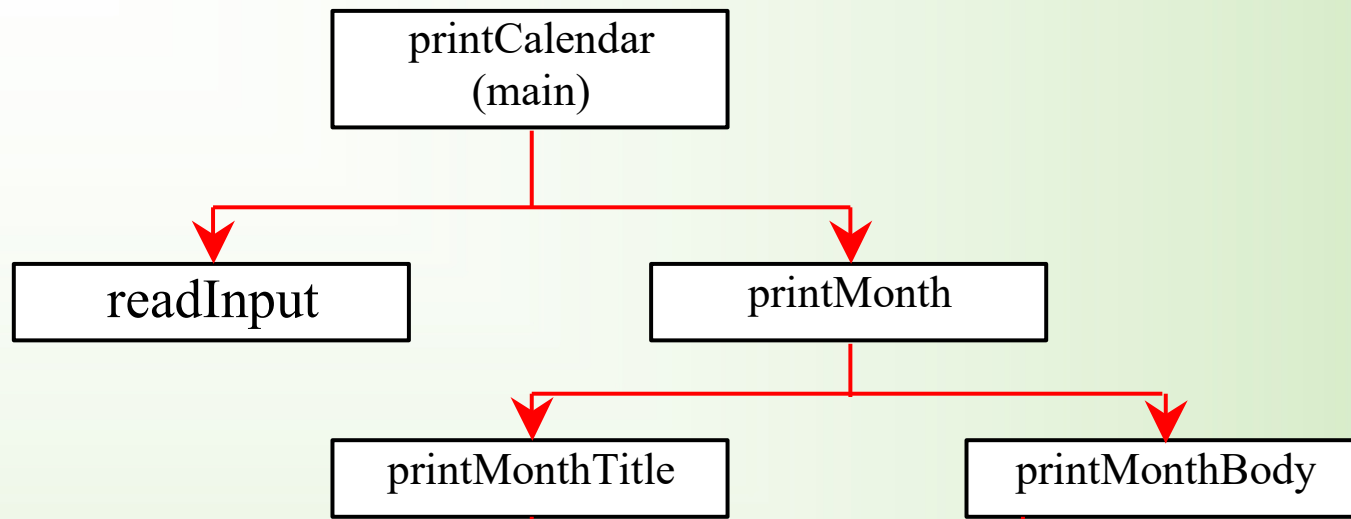


Design Diagram



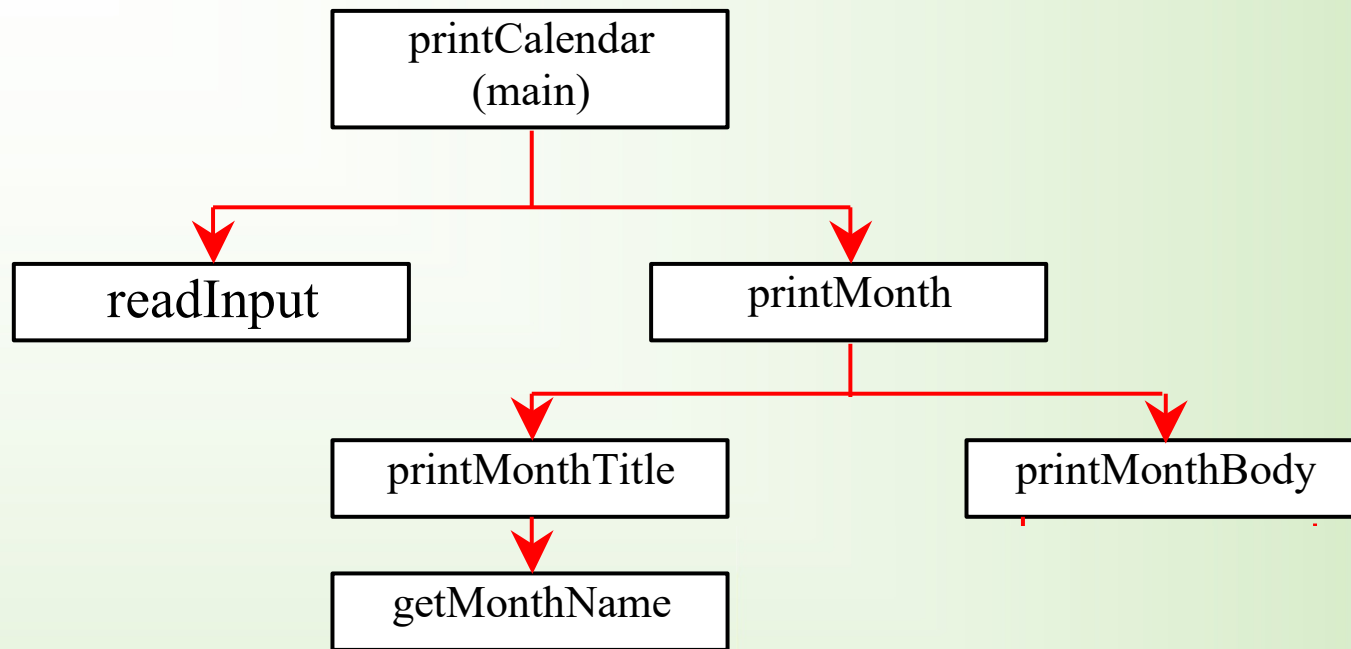


Design Diagram



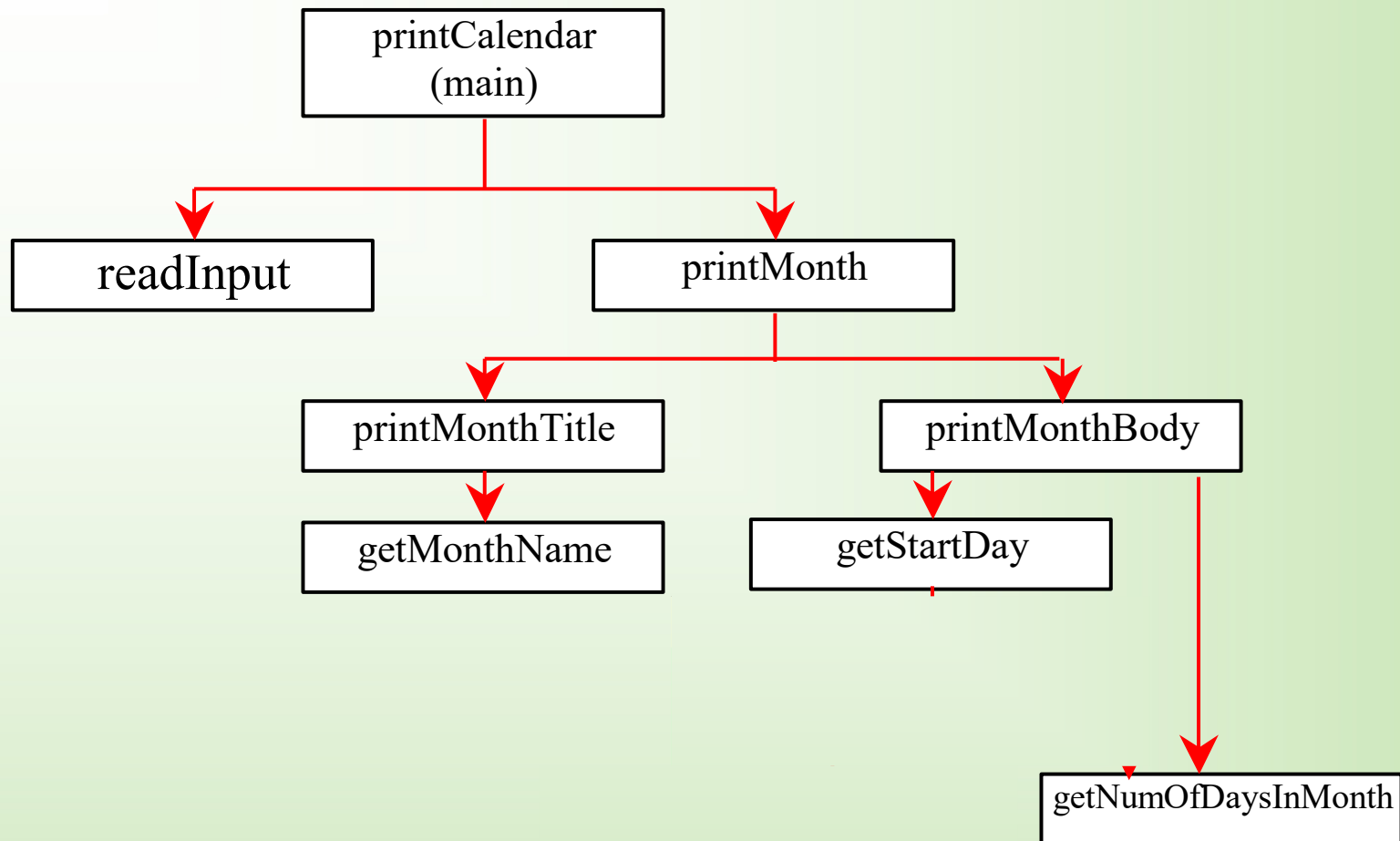


Design Diagram



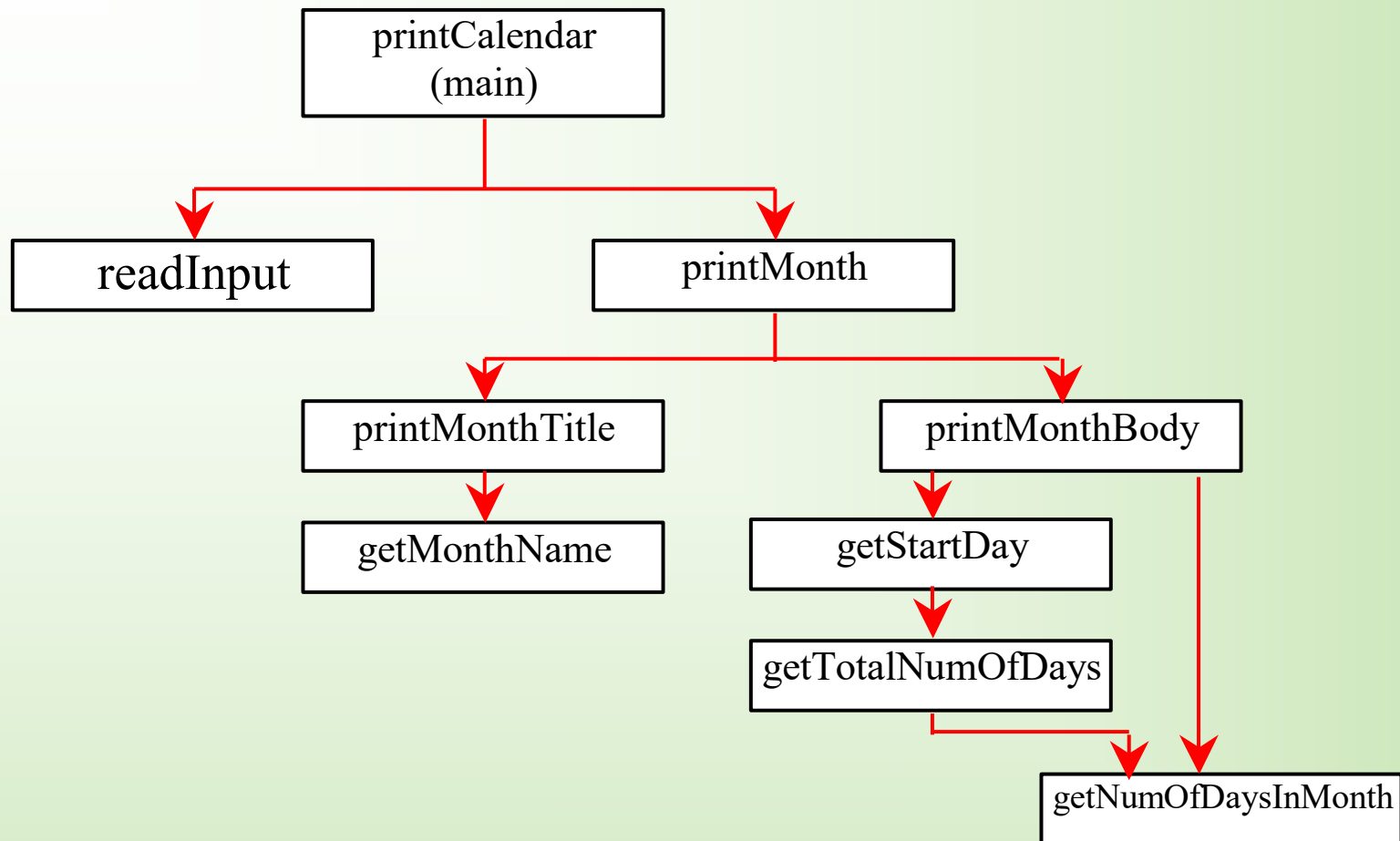


Design Diagram



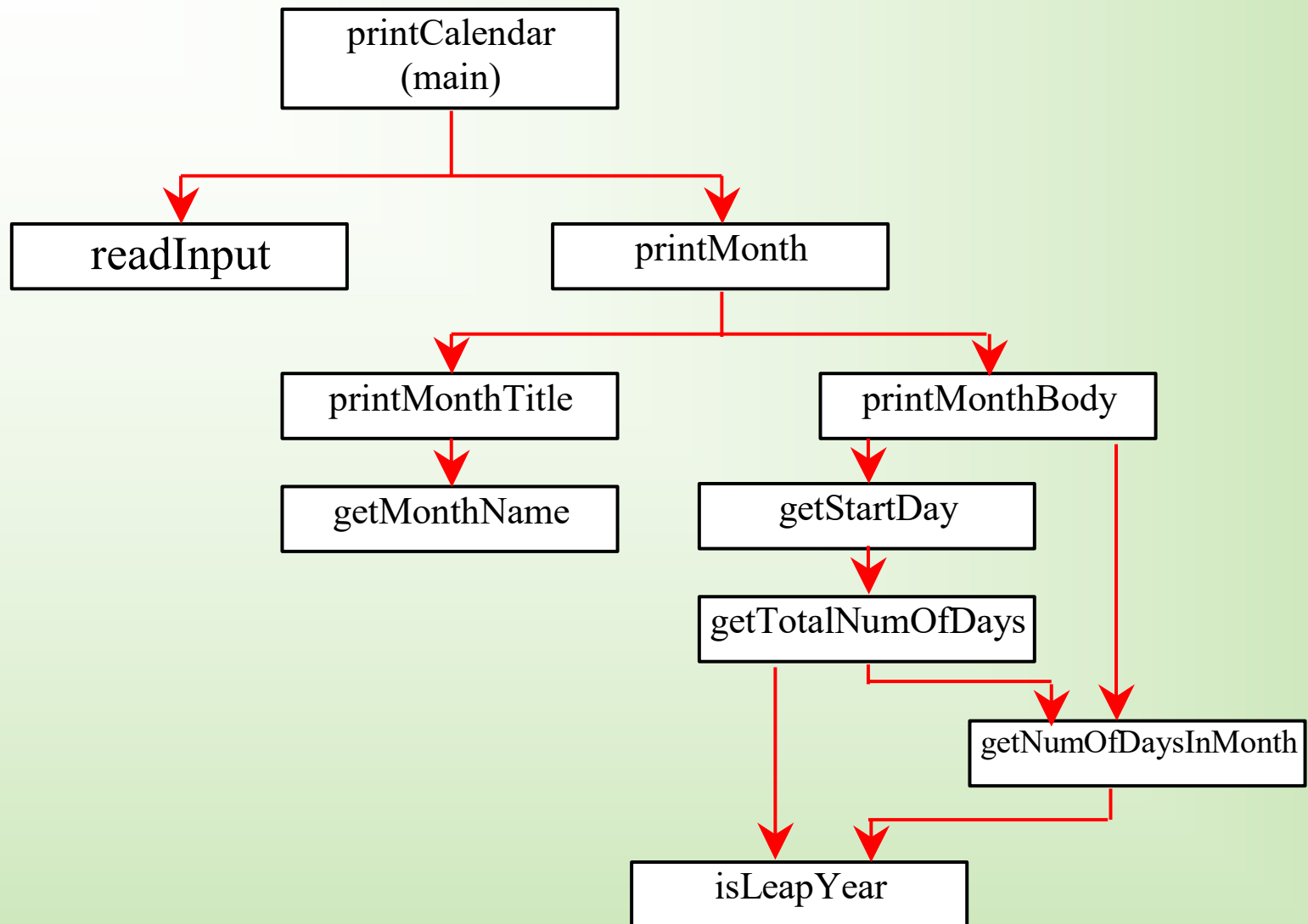


Design Diagram





Design Diagram





Implementation: Top-Down

♦ Step 1: Problem-solving Phase

- Top-down approach is to implement one method in the structure chart at a time from the top to the bottom.
 - ♦ Stubs can be used for the methods waiting to be implemented.
 - ♦ A stub is a simple but incomplete version of a method.
 - ♦ The use of stubs enables you to test invoking the method from a caller.
 - ♦ Implement the main method first and then use a stub for the printMonth method and so on

[Click here](#) to view a skeleton of the code

Program 8: PrintCalendar

♦ Run the Program:



Enter full year (e.g., 2012): 2012

Enter month as number between 1 and 12: 3

March 2012

Sun	Mon	Tue	Wed	Thu	Fri	Sat
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	

[Click here](#) to view and trace code



Benefits of Stepwise Refinement

- ♦ Some programs can be very long
 - This last program was only 100 lines, but it had several logically independent components
- ♦ Stepwise refinement breaks the larger problem down into smaller, more manageable subproblems
- ♦ Each subproblem can be implemented using a method
- ♦ This approach makes the program easier to:
 - write, reuse, debug, test, modify, and maintain



Benefits of Stepwise Refinement

♦ Reusing methods:

- Stepwise refinement encourages code reuse
- The **isLeapYear** method is defined once
- However, it is used twice:
 - ♦ Inside the **getTotalNumberOfDays**
 - ♦ Inside the **getNumberOfDayInMonth**



Benefits of Stepwise Refinement

- ♦ Easier developing, debugging, and testing
 - Each subproblem is developed in a method
 - This means each subproblem can be developed, debugged, and tested independently of other components of the problem
 - This isolates errors
 - Whenever you develop large programs, use this stepwise refinement approach
 - It may seem to take longer at first
 - But it saves time and makes debugging much easier!

Chapter 6:

Methods

